

Grafica

Generado por Doxygen 1.13.2

1	Índice jerárquico	1
1.1	Jerarquía de clases	1
2	Índice de clases	3
2.1	Lista de clases	3
3	Índice de archivos	5
3.1	Lista de archivos	5
4	Documentación de clases	7
4.1	Referencia de la clase <code>udit::AssimpMesh</code>	7
4.1.1	Descripción detallada	8
4.1.2	Documentación de constructores y destructores	8
4.1.2.1	<code>AssimpMesh()</code>	8
4.2	Referencia de la clase <code>udit::Camera</code>	9
4.2.1	Descripción detallada	9
4.2.2	Documentación de constructores y destructores	9
4.2.2.1	<code>Camera()</code>	9
4.2.3	Documentación de funciones miembro	10
4.2.3.1	<code>get_position()</code>	10
4.2.3.2	<code>get_view_matrix()</code>	10
4.2.3.3	<code>process_keyboard()</code>	10
4.2.3.4	<code>process_mouse_motion()</code>	10
4.2.3.5	<code>start_camera_control()</code>	11
4.3	Referencia de la clase <code>udit::Cone</code>	11
4.3.1	Descripción detallada	12
4.3.2	Documentación de constructores y destructores	12
4.3.2.1	<code>Cone()</code>	12
4.3.2.2	<code>~Cone()</code>	13
4.4	Referencia de la clase <code>udit::GeometryGenerator</code>	13
4.4.1	Descripción detallada	13
4.4.2	Documentación de funciones miembro	13
4.4.2.1	<code>generateCone()</code>	13
4.4.2.2	<code>generateCube()</code>	14
4.4.2.3	<code>generatePlane()</code>	14
4.5	Referencia de la clase <code>udit::Mesh</code>	14
4.5.1	Descripción detallada	15
4.5.2	Documentación de constructores y destructores	15
4.5.2.1	<code>Mesh()</code>	15
4.5.3	Documentación de funciones miembro	16
4.5.3.1	<code>deleteBuffers()</code>	16
4.5.3.2	<code>generateBuffers()</code>	16
4.5.3.3	<code>render()</code>	16
4.5.3.4	<code>uploadBuffer()</code>	16

4.6 Referencia de la estructura <code>udit::MeshData</code>	17
4.6.1 Descripción detallada	17
4.6.2 Documentación de datos miembro	17
4.6.2.1 <code>coordinates</code>	17
4.6.2.2 <code>indices</code>	17
4.6.2.3 <code>normals</code>	17
4.6.2.4 <code>texCoords</code>	17
4.7 Referencia de la clase <code>udit::Object</code>	18
4.7.1 Descripción detallada	18
4.7.2 Documentación de constructores y destructores	18
4.7.2.1 <code>Object()</code>	18
4.7.3 Documentación de funciones miembro	19
4.7.3.1 <code>getMesh()</code>	19
4.7.3.2 <code>getShader()</code>	19
4.7.3.3 <code>render()</code>	19
4.7.3.4 <code>setHeightmapTextureID()</code>	19
4.7.3.5 <code>setPosition()</code>	20
4.7.3.6 <code>setRotation()</code>	20
4.7.3.7 <code>setScale()</code>	20
4.7.3.8 <code>setTextureID()</code>	20
4.8 Referencia de la estructura <code>udit::Window::OpenGL_Context_Settings</code>	21
4.8.1 Descripción detallada	21
4.8.2 Documentación de datos miembro	21
4.8.2.1 <code>core_profile</code>	21
4.8.2.2 <code>depth_buffer_size</code>	21
4.8.2.3 <code>enable_vsync</code>	22
4.8.2.4 <code>stencil_buffer_size</code>	22
4.8.2.5 <code>version_major</code>	22
4.8.2.6 <code>version_minor</code>	22
4.9 Referencia de la estructura <code>Window::OpenGL_Context_Settings</code>	22
4.9.1 Descripción detallada	22
4.9.2 Documentación de datos miembro	23
4.9.2.1 <code>core_profile</code>	23
4.9.2.2 <code>depth_buffer_size</code>	23
4.9.2.3 <code>enable_vsync</code>	23
4.9.2.4 <code>stencil_buffer_size</code>	23
4.9.2.5 <code>version_major</code>	23
4.9.2.6 <code>version_minor</code>	23
4.10 Referencia de la clase <code>udit::Plane</code>	24
4.10.1 Descripción detallada	25
4.10.2 Documentación de constructores y destructores	25
4.10.2.1 <code>Plane()</code>	25
4.10.2.2 <code>~Plane()</code>	25

4.11 Referencia de la clase Scene	26
4.11.1 Descripción detallada	26
4.11.2 Documentación de constructores y destructores	26
4.11.2.1 Scene()	26
4.11.3 Documentación de funciones miembro	27
4.11.3.1 lightSetup()	27
4.11.3.2 resize()	27
4.11.3.3 set_camera()	27
4.12 Referencia de la clase udit::Scene	27
4.12.1 Descripción detallada	28
4.12.2 Documentación de constructores y destructores	28
4.12.2.1 Scene()	28
4.12.3 Documentación de funciones miembro	28
4.12.3.1 lightSetup()	28
4.12.3.2 resize()	29
4.12.3.3 set_camera()	29
4.13 Referencia de la clase udit::SceneNode	29
4.13.1 Descripción detallada	30
4.13.2 Documentación de constructores y destructores	30
4.13.2.1 SceneNode()	30
4.13.3 Documentación de funciones miembro	30
4.13.3.1 addChild()	30
4.13.3.2 render()	30
4.13.3.3 setTransform()	31
4.14 Referencia de la clase udit::ShaderProgram	32
4.14.1 Descripción detallada	32
4.14.2 Documentación de constructores y destructores	33
4.14.2.1 ShaderProgram() [1/3]	33
4.14.2.2 ShaderProgram() [2/3]	33
4.14.2.3 ShaderProgram() [3/3]	33
4.14.3 Documentación de funciones miembro	33
4.14.3.1 id()	33
4.14.3.2 operator=() [1/2]	33
4.14.3.3 operator=() [2/2]	33
4.14.3.4 setFloat()	34
4.14.3.5 setInt()	34
4.14.3.6 setMat4()	34
4.14.3.7 setVec3()	34
4.15 Referencia de la clase udit::Skybox	35
4.15.1 Descripción detallada	35
4.15.2 Documentación de constructores y destructores	35
4.15.2.1 Skybox()	35
4.15.3 Documentación de funciones miembro	36

4.15.3.1	getTextureID()	36
4.15.3.2	render()	36
4.15.3.3	setTexture()	36
4.16	Referencia de la clase <code>udit::TextureLoader</code>	37
4.16.1	Descripción detallada	37
4.16.2	Documentación de constructores y destructores	37
4.16.2.1	<code>TextureLoader()</code>	37
4.16.2.2	<code>~TextureLoader()</code>	37
4.16.3	Documentación de funciones miembro	37
4.16.3.1	<code>loadCubemap()</code>	37
4.16.3.2	<code>loadTexture()</code>	38
4.17	Referencia de la clase <code>udit::Window</code>	38
4.17.1	Descripción detallada	39
4.17.2	Documentación de las enumeraciones miembro de la clase	39
4.17.2.1	<code>Position</code>	39
4.17.3	Documentación de funciones miembro	39
4.17.3.1	<code>get_camera()</code>	39
4.17.3.2	<code>move_camera()</code>	40
4.17.3.3	<code>poll_input_events()</code>	41
4.17.3.4	<code>swap_buffers()</code>	41
4.18	Referencia de la clase <code>Window</code>	41
4.18.1	Descripción detallada	42
4.18.2	Documentación de las enumeraciones miembro de la clase	42
4.18.2.1	<code>Position</code>	42
4.18.3	Documentación de funciones miembro	43
4.18.3.1	<code>get_camera()</code>	43
4.18.3.2	<code>move_camera()</code>	43
4.18.3.3	<code>poll_input_events()</code>	43
4.18.3.4	<code>swap_buffers()</code>	43
5	Documentación de archivos	45
5.1	<code>AssimpMesh.hpp</code>	45
5.2	<code>Camera.hpp</code>	45
5.3	<code>Cone.hpp</code>	46
5.4	<code>GeometryGenerator.hpp</code>	46
5.5	<code>Mesh.hpp</code>	47
5.6	<code>Object.hpp</code>	47
5.7	<code>Plane.hpp</code>	48
5.8	Referencia del archivo <code>Scene.hpp</code>	48
5.8.1	Descripción detallada	49
5.9	<code>Scene.hpp</code>	49
5.10	<code>SceneNode.hpp</code>	50
5.11	<code>ShaderProgram.hpp</code>	51

5.12 Skybox.hpp	51
5.13 TextureLoader.hpp	52
5.14 Window.hpp	52

Capítulo 1

Índice de espacios de nombres

1.1. Lista de espacios de nombres

Lista de los espacios de nombres documentados, con breves descripciones:

<code>udit</code>		??
-------------------	-------	----

Capítulo 2

Índice jerárquico

2.1. Jerarquía de clases

Este listado de herencia está ordenado de forma general pero no está en orden alfabético estricto:

udit::Camera	9
udit::GeometryGenerator	13
udit::Mesh	14
udit::AssimpMesh	7
udit::Cone	11
udit::Plane	24
udit::MeshData	17
udit::Object	18
udit::Window::OpenGL_Context_Settings	21
Window::OpenGL_Context_Settings	22
Scene	26
udit::Scene	27
udit::SceneNode	29
udit::ShaderProgram	32
udit::Skybox	35
udit::TextureLoader	37
udit::Window	38
Window	41

Capítulo 3

Índice de clases

3.1. Lista de clases

Lista de clases, estructuras, uniones e interfaces con breves descripciones:

udit::AssimpMesh	7
Clase que extiende Mesh para cargar modelos 3D usando Assimp	
udit::Camera	9
Clase que representa una cámara en un espacio 3D, controlada mediante teclado y ratón	
udit::Cone	11
Representa un cono 3D con un número de divisiones, radio y altura	
udit::GeometryGenerator	13
Clase para generar primitivas geométricas básicas	
udit::Mesh	14
Clase que representa una malla 3D y gestiona sus buffers OpenGL	
udit::MeshData	17
Estructura que contiene datos de una malla generada	
udit::Object	18
Representa un objeto 3D con una malla, shader y transformaciones	
udit::Window::OpenGL_Context_Settings	21
Estructura para configurar los parámetros del contexto de OpenGL	
Window::OpenGL_Context_Settings	22
Estructura para configurar los parámetros del contexto de OpenGL	
udit::Plane	24
Clase para representar un plano 3D	
Scene	26
Representa una escena 3D con cámara, objetos, skybox, texturas, iluminación y shaders	
udit::Scene	27
Representa una escena 3D con cámara, objetos, skybox, texturas, iluminación y shaders	
udit::SceneNode	29
Nodo de escena para jerarquía de objetos 3D	
udit::ShaderProgram	32
Clase para manejar programas de sombreado (shaders) en OpenGL	
udit::Skybox	35
Clase que representa un skybox para el entorno 3D	
udit::TextureLoader	37
Clase para cargar texturas 2D y cubemaps en OpenGL	
udit::Window	38
Clase que encapsula una ventana con soporte para contexto OpenGL y control de cámara	
Window	41
Clase que encapsula una ventana con soporte para contexto OpenGL y control de cámara	

Capítulo 4

Índice de archivos

4.1. Lista de archivos

Lista de todos los archivos con breves descripciones:

AssimpMesh.hpp	??
Camera.hpp	??
Cone.hpp	??
GeometryGenerator.hpp	??
Mesh.hpp	??
Object.hpp	??
Plane.hpp	??
Scene.hpp	
Declaración de la clase Scene que representa una escena 3D completa	48
SceneNode.hpp	??
ShaderProgram.hpp	??
Skybox.hpp	??
TextureLoader.hpp	??
Window.hpp	??
AssimpMesh.cpp	??
Camera.cpp	??
Cone.cpp	??
GeometryGenerator.cpp	??
main.cpp	??
Mesh.cpp	??
Object.cpp	??
Plane.cpp	??
Scene.cpp	??
SceneNode.cpp	??
ShaderProgram.cpp	??
Skybox.cpp	??
stb_image.cpp	??
TextureLoader.cpp	??
Window.cpp	??

Capítulo 5

Documentación de espacios de nombres

5.1. Referencia del espacio de nombres udit

Clases

- class [AssimpMesh](#)
Clase que extiende [Mesh](#) para cargar modelos 3D usando Assimp.
- class [Camera](#)
Clase que representa una cámara en un espacio 3D, controlada mediante teclado y ratón.
- class [Cone](#)
Representa un cono 3D con un número de divisiones, radio y altura.
- class [GeometryGenerator](#)
Clase para generar primitivas geométricas básicas.
- class [Mesh](#)
Clase que representa una malla 3D y gestiona sus buffers OpenGL.
- struct [MeshData](#)
Estructura que contiene datos de una malla generada.
- class [Object](#)
Representa un objeto 3D con una malla, shader y transformaciones.
- class [Plane](#)
Clase para representar un plano 3D.
- class [Scene](#)
Representa una escena 3D con cámara, objetos, skybox, texturas, iluminación y shaders.
- class [SceneNode](#)
Nodo de escena para jerarquía de objetos 3D.
- class [ShaderProgram](#)
Clase para manejar programas de sombreado (shaders) en OpenGL.
- class [Skybox](#)
Clase que representa un skybox para el entorno 3D.
- class [TextureLoader](#)
Clase para cargar texturas 2D y cubemaps en OpenGL.
- class [Window](#)
Clase que encapsula una ventana con soporte para contexto OpenGL y control de cámara.

Funciones

- static void [addVertex](#) ([MeshData](#) &data, const glm::vec3 &pos, const glm::vec3 &normal, const glm::vec2 &uv)

5.1.1. Documentación de funciones

5.1.1.1. addVertex()

```
static void udit::addVertex (  
    MeshData & data,  
    const glm::vec3 & pos,  
    const glm::vec3 & normal,  
    const glm::vec2 & uv) [static]
```

Capítulo 6

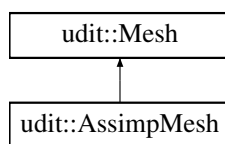
Documentación de clases

6.1. Referencia de la clase `udit::AssimpMesh`

Clase que extiende [Mesh](#) para cargar modelos 3D usando Assimp.

```
#include <AssimpMesh.hpp>
```

Diagrama de herencia de `udit::AssimpMesh`



Métodos públicos

- [AssimpMesh](#) (const std::string &filepath)
Constructor que carga un modelo desde un archivo.

Métodos públicos heredados de [udit::Mesh](#)

- [Mesh](#) ()
Constructor por defecto. Inicializa la malla vacía y genera los buffers OpenGL.
- [Mesh](#) ([MeshData](#) receivedData)
Constructor con datos de malla.
- void [render](#) ()
Renderiza la malla usando los datos almacenados y buffers GPU.
- void [generateBuffers](#) ()
Genera los buffers OpenGL (VBOs y VAO) para la malla actual.
- void [uploadBuffer](#) (GLuint bufferId, GLenum target, const void *dataPtr, size_t dataSize)
Carga datos en un buffer de OpenGL.
- void [deleteBuffers](#) ()
Elimina los buffers OpenGL asociados a esta malla.

Métodos privados

- void [loadModel](#) (const std::string &path)
Carga el modelo desde disco usando Assimp.
- void [processMesh](#) (aiMesh *mesh)
Procesa una malla de Assimp extrayendo vértices, normales, UVs e índices.
- void [processVertices](#) (aiMesh *mesh)
Procesa los vértices de la malla, incluyendo coordenadas, normales y UVs.
- void [processIndices](#) (aiMesh *mesh)
Procesa los índices de la malla para formar los triángulos.

Otros miembros heredados

Tipos protegidos heredados de [udit::Mesh](#)

- enum {
[COORDINATES_VBO](#) , [TEXCOORDS_VBO](#) , [NORMALS_VBO](#) , [INDICES_EBO](#) ,
[VBO_COUNT](#) }
Índices para los diferentes buffers usados.

Atributos protegidos heredados de [udit::Mesh](#)

- bool [isInitialized](#)
Indica si los buffers están inicializados y listos para renderizar.
- GLuint [vao_id](#)
Identificador del Vertex Array [Object](#).
- GLuint [vbo_ids](#) [[VBO_COUNT](#)]
Identificadores de Vertex Buffer Objects y Element Buffer [Object](#).
- [MeshData](#) [data](#)
Datos de la malla: coordenadas, normales, texturas e índices.

6.1.1. Descripción detallada

Clase que extiende [Mesh](#) para cargar modelos 3D usando Assimp.

Esta clase carga un modelo desde un archivo usando Assimp y genera los buffers necesarios para renderizar la primera malla encontrada.

6.1.2. Documentación de constructores y destructores

6.1.2.1. [AssimpMesh](#)()

```
udit::AssimpMesh::AssimpMesh (  
    const std::string & filepath)
```

Constructor que carga un modelo desde un archivo.

Parámetros

filepath	Ruta al archivo del modelo 3D.
----------	--------------------------------

6.1.3. Documentación de funciones miembro

6.1.3.1. loadModel()

```
void udit::AssimpMesh::loadModel (
    const std::string & path) [private]
```

Carga el modelo desde disco usando Assimp.

Parámetros

path	Ruta al archivo del modelo 3D.
------	--------------------------------

Lee la escena completa y procesa la primera malla que encuentre.

6.1.3.2. processIndices()

```
void udit::AssimpMesh::processIndices (
    aiMesh * mesh) [private]
```

Procesa los índices de la malla para formar los triángulos.

Parámetros

mesh	Puntero a la malla de Assimp.
------	-------------------------------

6.1.3.3. processMesh()

```
void udit::AssimpMesh::processMesh (
    aiMesh * mesh) [private]
```

Procesa una malla de Assimp extrayendo vértices, normales, UVs e índices.

Parámetros

mesh	Puntero a la malla de Assimp a procesar.
------	--

6.1.3.4. processVertices()

```
void udit::AssimpMesh::processVertices (
    aiMesh * mesh) [private]
```

Procesa los vértices de la malla, incluyendo coordenadas, normales y UVs.

Parámetros

mesh	Puntero a la malla de Assimp.
------	-------------------------------

La documentación de esta clase está generada de los siguientes archivos:

- [AssimpMesh.hpp](#)
- [AssimpMesh.cpp](#)

6.2. Referencia de la clase `udit::Camera`

Clase que representa una cámara en un espacio 3D, controlada mediante teclado y ratón.

```
#include <Camera.hpp>
```

Métodos públicos

- [Camera](#) (`glm::vec3 start_position, glm::vec3 start_up, float start_yaw, float start_pitch`)
Constructor con parámetros para inicializar la cámara con valores específicos.
- [Camera](#) ()
Constructor por defecto. Inicializa la cámara con valores neutros.
- `glm::mat4` [get_view_matrix](#) () const
Genera y retorna la matriz de vista de la cámara.
- `glm::vec3` [get_position](#) () const
Obtiene la posición actual de la cámara en el espacio.
- void [process_keyboard](#) (const `UInt8 *state`)
Procesa la entrada del teclado para mover la cámara.
- void [start_camera_control](#) ()
Configura el control de la cámara activando el modo de ratón relativo.
- void [process_mouse_motion](#) (float `xrel`, float `yrel`)
Procesa el movimiento del ratón para rotar la cámara.

Métodos privados

- void [update_camera_vectors](#) ()
Actualiza los vectores de orientación de la cámara (`front`, `right`, `up`).

Atributos privados

- `glm::vec3` [position](#)
- `glm::vec3` [front](#)
- `glm::vec3` [up](#)
- `glm::vec3` [right](#)
- `glm::vec3` [world_up](#)
- float [yaw](#)
- float [pitch](#)
- float [movement_speed](#)
- float [mouse_sensitivity](#)

6.2.1. Descripción detallada

Clase que representa una cámara en un espacio 3D, controlada mediante teclado y ratón.

Permite calcular la matriz de vista y moverse/orientarse en un entorno tridimensional usando SDL.

6.2.2. Documentación de constructores y destructores

6.2.2.1. Camera() [1/2]

```
udit::Camera::Camera (
    glm::vec3 start_position,
    glm::vec3 start_up,
    float start_yaw,
    float start_pitch)
```

Constructor con parámetros para inicializar la cámara con valores específicos.

Parámetros

start_position	Posición inicial de la cámara.
start_up	Vector de orientación inicial hacia arriba.
start_yaw	Valor inicial del ángulo yaw.
start_pitch	Valor inicial del ángulo pitch.

6.2.2.2. Camera() [2/2]

```
udit::Camera::Camera ()
```

Constructor por defecto. Inicializa la cámara con valores neutros.

6.2.3. Documentación de funciones miembro

6.2.3.1. get_position()

```
glm::vec3 udit::Camera::get_position () const
```

Obtiene la posición actual de la cámara en el espacio.

Devuelve

glm::vec3 Posición actual.

6.2.3.2. get_view_matrix()

```
glm::mat4 udit::Camera::get_view_matrix () const
```

Genera y retorna la matriz de vista de la cámara.

Devuelve

glm::mat4 Matriz de vista utilizada para transformar el mundo en coordenadas de cámara.

6.2.3.3. process_keyboard()

```
void udit::Camera::process_keyboard (
    const Uint8 * state)
```

Procesa la entrada del teclado para mover la cámara.

Parámetros

state	Puntero al estado actual del teclado proporcionado por <code>SDL_GetKeyboardState()</code> .
-------	--

6.2.3.4. `process_mouse_motion()`

```
void udit::Camera::process_mouse_motion (
    float xrel,
    float yrel)
```

Procesa el movimiento del ratón para rotar la cámara.

Parámetros

xrel	Movimiento relativo en el eje X del ratón.
yrel	Movimiento relativo en el eje Y del ratón.

6.2.3.5. `start_camera_control()`

```
void udit::Camera::start_camera_control ()
```

Configura el control de la cámara activando el modo de ratón relativo.

Oculto el cursor y permite capturar el movimiento del ratón.

6.2.3.6. `update_camera_vectors()`

```
void udit::Camera::update_camera_vectors () [private]
```

Actualiza los vectores de orientación de la cámara (front, right, up).

Este método debe llamarse cada vez que cambien los ángulos yaw o pitch.

6.2.4. Documentación de datos miembro

6.2.4.1. `front`

```
glm::vec3 udit::Camera::front [private]
```

Vector de dirección hacia donde la cámara está mirando.

6.2.4.2. `mouse_sensitivity`

```
float udit::Camera::mouse_sensitivity [private]
```

Sensibilidad de rotación con el movimiento del ratón.

6.2.4.3. `movement_speed`

`float udit::Camera::movement_speed` [private]

Velocidad de desplazamiento de la cámara.

6.2.4.4. `pitch`

`float udit::Camera::pitch` [private]

Ángulo de rotación vertical (eje X).

6.2.4.5. `position`

`glm::vec3 udit::Camera::position` [private]

Posición actual de la cámara en el espacio 3D.

6.2.4.6. `right`

`glm::vec3 udit::Camera::right` [private]

Vector hacia la derecha relativo a la cámara.

6.2.4.7. `up`

`glm::vec3 udit::Camera::up` [private]

Vector hacia arriba relativo a la cámara.

6.2.4.8. `world_up`

`glm::vec3 udit::Camera::world_up` [private]

Vector global hacia arriba (normalmente (0,1,0)).

6.2.4.9. `yaw`

`float udit::Camera::yaw` [private]

Ángulo de rotación horizontal (eje Y).

La documentación de esta clase está generada de los siguientes archivos:

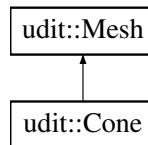
- [Camera.hpp](#)
- [Camera.cpp](#)

6.3. Referencia de la clase udit::Cone

Representa un cono 3D con un n mero de divisiones, radio y altura.

```
#include <Cone.hpp>
```

Diagrama de herencia de udit::Cone



Métodos públicos

- [Cone](#) (int d, GLfloat r, GLfloat h)
Constructor de la clase [Cone](#).
- [~Cone](#) ()
Destructor de la clase [Cone](#).

Métodos públicos heredados de [udit::Mesh](#)

- [Mesh](#) ()
Constructor por defecto. Inicializa la malla vacía y genera los buffers OpenGL.
- [Mesh](#) ([MeshData](#) receivedData)
Constructor con datos de malla.
- void [render](#) ()
Renderiza la malla usando los datos almacenados y buffers GPU.
- void [generateBuffers](#) ()
Genera los buffers OpenGL (VBOs y VAO) para la malla actual.
- void [uploadBuffer](#) (GLuint bufferId, GLenum target, const void *dataPtr, size_t dataSize)
Carga datos en un buffer de OpenGL.
- void [deleteBuffers](#) ()
Elimina los buffers OpenGL asociados a esta malla.

Métodos privados

- void [generateGeometry](#) ()
- void [generateApexVertex](#) (int &vertexIndex, int apexIndex)
- void [generateBaseCenterVertex](#) (int &vertexIndex, int baseCenterIndex)
- void [generateBaseVertices](#) (int &vertexIndex)

Atributos privados

- int [divisions](#)
- GLfloat [radius](#)
- GLfloat [height](#)

Otros miembros heredados

Tipos protegidos heredados de `udit::Mesh`

- enum {
`COORDINATES_VBO` , `TEXCOORDS_VBO` , `NORMALS_VBO` , `INDICES_EBO` ,
`VBO_COUNT` }
 Índices para los diferentes buffers usados.

Atributos protegidos heredados de `udit::Mesh`

- bool `isInitialized`
 Indica si los buffers están inicializados y listos para renderizar.
- GLuint `vao_id`
 Identificador del Vertex Array Object.
- GLuint `vbo_ids` [`VBO_COUNT`]
 Identificadores de Vertex Buffer Objects y Element Buffer Object.
- `MeshData data`
 Datos de la malla: coordenadas, normales, texturas e índices.

6.3.1. Descripción detallada

Representa un cono 3D con un número de divisiones, radio y altura.

La clase permite la creación de un cono 3D, la gestión de sus geometrías (coordenadas, índices y coordenadas de textura) y el renderizado del mismo utilizando OpenGL. El cono se genera a partir de las especificaciones de radio, altura y divisiones.

6.3.2. Documentación de constructores y destructores

6.3.2.1. `Cone()`

```
udit::Cone::Cone (
    int d,
    GLfloat r,
    GLfloat h)
```

Constructor de la clase `Cone`.

Inicializa el cono con el número de divisiones, el radio y la altura especificados. Este constructor también prepara las estructuras necesarias para el renderizado.

Parámetros

d	Número de divisiones del cono.
r	Radio de la base del cono.
h	Altura del cono.

6.3.2.2. `~Cone()`

```
udit::Cone::~Cone ()
```

Destructor de la clase [Cone](#).

Libera los recursos utilizados por los buffers de OpenGL (VAO y VBOs).

6.3.3. Documentación de funciones miembro

6.3.3.1. `generateApexVertex()`

```
void udit::Cone::generateApexVertex (
    int & vertexIndex,
    int apexIndex) [private]
```

6.3.3.2. `generateBaseCenterVertex()`

```
void udit::Cone::generateBaseCenterVertex (
    int & vertexIndex,
    int baseCenterIndex) [private]
```

6.3.3.3. `generateBaseVertices()`

```
void udit::Cone::generateBaseVertices (
    int & vertexIndex) [private]
```

6.3.3.4. `generateGeometry()`

```
void udit::Cone::generateGeometry () [private]
```

6.3.4. Documentación de datos miembro

6.3.4.1. `divisions`

```
int udit::Cone::divisions [private]
```

N mero de divisiones que se utilizar n para crear el cono.

6.3.4.2. `height`

```
GLfloat udit::Cone::height [private]
```

Altura del cono.

6.3.4.3. radius

GLfloat udit::Cone::radius [private]

Radio de la base del cono.

La documentación de esta clase está generada de los siguientes archivos:

- [Cone.hpp](#)
- [Cone.cpp](#)

6.4. Referencia de la clase udit::GeometryGenerator

Clase para generar primitivas geométricas básicas.

```
#include <GeometryGenerator.hpp>
```

Métodos públicos estáticos

- static [MeshData generatePlane](#) (int hDivisions, int vDivisions, float width, float height)
Genera una malla de un plano subdividido.
- static [MeshData generateCone](#) (float radius, float height, int segments)
Genera una malla de un cono.
- static [MeshData generateCube](#) (float size=1.0f)
Genera una malla de un cubo centrado en el origen.

6.4.1. Descripción detallada

Clase para generar primitivas geométricas básicas.

Proporciona funciones estáticas para crear mallas de planos, conos y cubos.

6.4.2. Documentación de funciones miembro

6.4.2.1. generateCone()

```
MeshData udit::GeometryGenerator::generateCone (
    float radius,
    float height,
    int segments) [static]
```

Genera una malla de un cono.

Parámetros

radius	Radio de la base del cono.
height	Altura del cono.
segments	Número de segmentos para aproximar la base circular.

Devuelve

[MeshData](#) con vértices, normales, UVs e índices.

6.4.2.2. generateCube()

[MeshData](#) `udit::GeometryGenerator::generateCube (`
 `float size = 1.0f) [static]`

Genera una malla de un cubo centrado en el origen.

Parámetros

size	Tamaño del lado del cubo. Por defecto 1.0f.
------	---

Devuelve

[MeshData](#) con vértices, normales, UVs e índices.

6.4.2.3. generatePlane()

```
MeshData udit::GeometryGenerator::generatePlane (
    int hDivisions,
    int vDivisions,
    float width,
    float height) [static]
```

Genera una malla de un plano subdividido.

Parámetros

hDivisions	Número de subdivisiones horizontales.
vDivisions	Número de subdivisiones verticales.
width	Ancho total del plano.
height	Alto total del plano.

Devuelve

[MeshData](#) con los datos de vértices, normales, UVs e índices.

La documentación de esta clase está generada de los siguientes archivos:

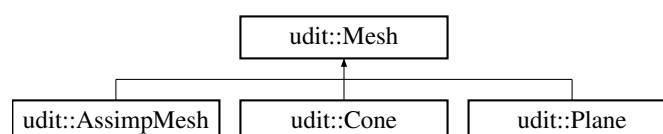
- [GeometryGenerator.hpp](#)
- [GeometryGenerator.cpp](#)

6.5. Referencia de la clase udit::Mesh

Clase que representa una malla 3D y gestiona sus buffers OpenGL.

```
#include <Mesh.hpp>
```

Diagrama de herencia de udit::Mesh



Métodos públicos

- [Mesh](#) ()
Constructor por defecto. Inicializa la malla vacía y genera los buffers OpenGL.
- [Mesh](#) ([MeshData](#) receivedData)
Constructor con datos de malla.
- void [render](#) ()
Renderiza la malla usando los datos almacenados y buffers GPU.
- void [generateBuffers](#) ()
Genera los buffers OpenGL (VBOs y VAO) para la malla actual.
- void [uploadBuffer](#) (GLuint bufferId, GLenum target, const void *dataPtr, size_t dataSize)
Carga datos en un buffer de OpenGL.
- void [deleteBuffers](#) ()
Elimina los buffers OpenGL asociados a esta malla.

Tipos protegidos

- enum {
 [COORDINATES_VBO](#) , [TEXCOORDS_VBO](#) , [NORMALS_VBO](#) , [INDICES_EBO](#) ,
 [VBO_COUNT](#) }
 Índices para los diferentes buffers usados.

Atributos protegidos

- bool [isInitialized](#)
Indica si los buffers están inicializados y listos para renderizar.
- GLuint [vao_id](#)
Identificador del Vertex Array [Object](#).
- GLuint [vbo_ids](#) [[VBO_COUNT](#)]
Identificadores de Vertex Buffer Objects y Element Buffer [Object](#).
- [MeshData](#) [data](#)
Datos de la malla: coordenadas, normales, texturas e índices.

6.5.1. Descripción detallada

Clase que representa una malla 3D y gestiona sus buffers OpenGL.

Esta clase encapsula los datos de la malla (vértices, normales, texturas, índices) y proporciona funciones para crear y liberar buffers GPU, así como para renderizar.

6.5.2. Documentación de las enumeraciones miembro de la clase

6.5.2.1. anonymous enum

anonymous enum [protected]

Índices para los diferentes buffers usados.

Valores de enumeraciones

COORDINATES_VBO	
TEXCOORDS_VBO	
NORMALS_VBO	
INDICES_EBO	
VBO_COUNT	

6.5.3. Documentación de constructores y destructores

6.5.3.1. Mesh() [1/2]

```
udit::Mesh::Mesh ()
```

Constructor por defecto. Inicializa la malla vacía y genera los buffers OpenGL.

6.5.3.2. Mesh() [2/2]

```
udit::Mesh::Mesh (
    MeshData receivedData)
```

Constructor con datos de malla.

Parámetros

receivedData	Datos de la malla (vértices, normales, texturas, índices). Crea la malla y genera los buffers en GPU con esos datos.
--------------	--

6.5.4. Documentación de funciones miembro

6.5.4.1. deleteBuffers()

```
void udit::Mesh::deleteBuffers ()
```

Elimina los buffers OpenGL asociados a esta malla.

Libera memoria GPU y marca la malla como no inicializada.

6.5.4.2. generateBuffers()

```
void udit::Mesh::generateBuffers ()
```

Genera los buffers OpenGL (VBOs y VAO) para la malla actual.

Carga los datos (vértices, normales, texturas e índices) a la GPU. Marca la malla como inicializada para renderizado posterior.

6.5.4.3. render()

```
void udit::Mesh::render ()
```

Renderiza la malla usando los datos almacenados y buffers GPU.

Si la malla no está inicializada, no hace nada. Usa `glDrawElements` con modo `GL_TRIANGLES`.

6.5.4.4. uploadBuffer()

```
void udit::Mesh::uploadBuffer (
    GLuint bufferId,
    GLenum target,
    const void * dataPtr,
    size_t dataSize)
```

Carga datos en un buffer de OpenGL.

Parámetros

bufferId	Identificador del buffer a configurar.
target	Tipo de buffer (por ejemplo, <code>GL_ARRAY_BUFFER</code> o <code>GL_ELEMENT_ARRAY_BUFFER</code>).
dataPtr	Puntero a los datos a subir al buffer.
dataSize	Tamaño en bytes de los datos.

6.5.5. Documentación de datos miembro

6.5.5.1. data

[MeshData](#) udit::Mesh::data [protected]

Datos de la malla: coordenadas, normales, texturas e índices.

6.5.5.2. isInitialized

bool udit::Mesh::isInitialized [protected]

Indica si los buffers están inicializados y listos para renderizar.

6.5.5.3. vao_id

GLuint udit::Mesh::vao_id [protected]

Identificador del Vertex Array [Object](#).

6.5.5.4. vbo_ids

GLuint udit::Mesh::vbo_ids[VBO_COUNT] [protected]

Identificadores de Vertex Buffer Objects y Element Buffer [Object](#).

La documentación de esta clase está generada de los siguientes archivos:

- [Mesh.hpp](#)
- [Mesh.cpp](#)

6.6. Referencia de la estructura udit::MeshData

Estructura que contiene datos de una malla generada.

```
#include <GeometryGenerator.hpp>
```

Atributos públicos

- std::vector< GLfloat > [coordinates](#)
- std::vector< GLfloat > [texCoords](#)
- std::vector< GLfloat > [normals](#)
- std::vector< GLuint > [indices](#)

6.6.1. Descripción detallada

Estructura que contiene datos de una malla generada.

Incluye coordenadas de vértices, coordenadas de textura, normales e índices.

6.6.2. Documentación de datos miembro

6.6.2.1. coordinates

std::vector<GLfloat> udit::MeshData::coordinates

Posiciones de vértices (x, y, z).

6.6.2.2. indices

std::vector<GLuint> udit::MeshData::indices

Índices para dibujar triángulos.

6.6.2.3. normals

`std::vector<GLfloat> udit::MeshData::normals`

Normales para iluminación (x, y, z).

6.6.2.4. texCoords

`std::vector<GLfloat> udit::MeshData::texCoords`

Coordenadas de textura (u, v).

La documentación de esta estructura está generada del siguiente archivo:

- [GeometryGenerator.hpp](#)

6.7. Referencia de la clase `udit::Object`

Representa un objeto 3D con una malla, shader y transformaciones.

`#include <Object.hpp>`

Métodos públicos

- [Object](#) ([Mesh](#) *mesh, [ShaderProgram](#) *shader)
 - Constructor que inicializa el objeto con una malla y un shader.
- void [render](#) (const glm::mat4 &view, const glm::mat4 &projection)
 - Renderiza el objeto usando las matrices de vista y proyección dadas.
- void [setupTexturesAndBlending](#) ()
 - Configura las texturas y el blending para la renderización, seleccionando el método adecuado según la presencia de heightmap o transparencia.
- void [bindHeightmapAndTexture](#) ()
 - Vincula la textura del heightmap y la textura principal, configurando los uniformes y activando las unidades de textura necesarias.
- void [bindTextureWithTransparency](#) ()
 - Vincula la textura principal y habilita el blending para soportar transparencia, ajustando el estado de profundidad y mezcla.
- void [bindTextureOnly](#) ()
 - Vincula solo la textura principal sin blending ni heightmap.
- void [cleanupBlending](#) ()
 - Limpia el estado de blending y máscara de profundidad después de renderizar objetos con transparencia.
- void [setPosition](#) (const glm::vec3 &pos)
 - Establece la posición del objeto en el espacio 3D.
- void [setRotation](#) (const glm::vec3 &rot)
 - Establece la rotación del objeto en grados sobre cada eje.
- void [setScale](#) (const glm::vec3 &scl)
 - Establece la escala del objeto en cada eje.
- void [setTextureID](#) (GLuint texture)
 - Asigna la textura principal (color) del objeto.
- void [setHeightmapTextureID](#) (GLuint texture)
 - Asigna la textura del mapa de altura para efectos de desplazamiento.
- [ShaderProgram](#) * [getShader](#) () const
 - Obtiene el shader asociado al objeto.
- [Mesh](#) * [getMesh](#) () const
 - Obtiene la malla asociada al objeto.

Métodos privados

- glm::mat4 [computeModelMatrix](#) () const
Calcula la matriz modelo combinando traslación, rotación y escala.

Atributos privados

- [Mesh](#) * [mesh](#)
Puntero a la malla del objeto.
- [ShaderProgram](#) * [shader](#)
Puntero al programa de shader utilizado.
- GLuint [textureID](#)
ID de la textura principal.
- GLuint [heightmapID](#)
ID de la textura del mapa de altura.
- GLint [heightmapLoc](#)
Localización del uniform "heightmap" en el shader.
- GLint [transparencyLoc](#)
Localización del uniform "transparency" en el shader.
- glm::vec3 [position](#)
Posición del objeto en el espacio 3D.
- glm::vec3 [rotation](#)
Rotación del objeto (en grados) sobre cada eje.
- glm::vec3 [scale](#)
Escala del objeto en cada eje.

6.7.1. Descripción detallada

Representa un objeto 3D con una malla, shader y transformaciones.

Permite configurar posición, rotación, escala, texturas y renderizar el objeto.

6.7.2. Documentación de constructores y destructores

6.7.2.1. Object()

```
udit::Object::Object (
    Mesh * mesh,
    ShaderProgram * shader)
```

Constructor que inicializa el objeto con una malla y un shader.

Parámetros

mesh	Puntero a la malla a renderizar.
shader	Puntero al programa de shader para renderizar el objeto.

6.7.3. Documentación de funciones miembro

6.7.3.1. bindHeightmapAndTexture()

```
void udit::Object::bindHeightmapAndTexture ()
```

Vincula la textura del heightmap y la textura principal, configurando los uniformes y activando las unidades de textura necesarias.

6.7.3.2. bindTextureOnly()

```
void udit::Object::bindTextureOnly ()
```

Vincula solo la textura principal sin blending ni heightmap.

6.7.3.3. bindTextureWithTransparency()

```
void udit::Object::bindTextureWithTransparency ()
```

Vincula la textura principal y habilita el blending para soportar transparencia, ajustando el estado de profundidad y mezcla.

6.7.3.4. cleanupBlending()

```
void udit::Object::cleanupBlending ()
```

Limpia el estado de blending y máscara de profundidad después de renderizar objetos con transparencia.

6.7.3.5. computeModelMatrix()

```
glm::mat4 udit::Object::computeModelMatrix () const [private]
```

Calcula la matriz modelo combinando traslación, rotación y escala.

Devuelve

Matriz modelo 4x4.

6.7.3.6. getMesh()

```
Mesh * udit::Object::getMesh () const
```

Obtiene la malla asociada al objeto.

Devuelve

Puntero a la malla.

6.7.3.7. getShader()

```
ShaderProgram * udit::Object::getShader () const
```

Obtiene el shader asociado al objeto.

Devuelve

Puntero al [ShaderProgram](#).

6.7.3.8. render()

```
void udit::Object::render (  
    const glm::mat4 & view,  
    const glm::mat4 & projection)
```

Renderiza el objeto usando las matrices de vista y proyección dadas.

Configura texturas, activa blending si es necesario, y envía las matrices al shader.

Parámetros

view	Matriz de vista (camera).
projection	Matriz de proyección.

6.7.3.9. setHeightmapTextureID()

```
void udit::Object::setHeightmapTextureID (  
    GLuint texture)
```

Asigna la textura del mapa de altura para efectos de desplazamiento.

Parámetros

texture	ID de la textura OpenGL para el mapa de altura.
---------	---

6.7.3.10. setPosition()

```
void udit::Object::setPosition (  
    const glm::vec3 & pos)
```

Establece la posición del objeto en el espacio 3D.

Parámetros

pos	Vector 3D con la nueva posición.
-----	----------------------------------

6.7.3.11. setRotation()

```
void udit::Object::setRotation (  
    const glm::vec3 & rot)
```

Establece la rotación del objeto en grados sobre cada eje.

Parámetros

rot	Vector 3D con la rotación en grados (x, y, z).
-----	--

6.7.3.12. setScale()

```
void udit::Object::setScale (
    const glm::vec3 & scl)
```

Establece la escala del objeto en cada eje.

Parámetros

scl	Vector 3D con los factores de escala.
-----	---------------------------------------

6.7.3.13. setTextureID()

```
void udit::Object::setTextureID (
    GLuint texture)
```

Asigna la textura principal (color) del objeto.

Parámetros

texture	ID de la textura OpenGL.
---------	--------------------------

6.7.3.14. setupTexturesAndBlending()

```
void udit::Object::setupTexturesAndBlending ()
```

Configura las texturas y el blending para la renderización, seleccionando el método adecuado según la presencia de heightmap o transparencia.

6.7.4. Documentación de datos miembro

6.7.4.1. heightmapID

```
GLuint udit::Object::heightmapID [private]
```

ID de la textura del mapa de altura.

6.7.4.2. heightmapLoc

```
GLint udit::Object::heightmapLoc [private]
```

Localización del uniform "heightmap" en el shader.

6.7.4.3. `mesh`

[Mesh*](#) `udit::Object::mesh` [private]

Puntero a la malla del objeto.

6.7.4.4. `position`

`glm::vec3` `udit::Object::position` [private]

Posición del objeto en el espacio 3D.

6.7.4.5. `rotation`

`glm::vec3` `udit::Object::rotation` [private]

Rotación del objeto (en grados) sobre cada eje.

6.7.4.6. `scale`

`glm::vec3` `udit::Object::scale` [private]

Escala del objeto en cada eje.

6.7.4.7. `shader`

[ShaderProgram*](#) `udit::Object::shader` [private]

Puntero al programa de shader utilizado.

6.7.4.8. `textureID`

`GLuint` `udit::Object::textureID` [private]

ID de la textura principal.

6.7.4.9. `transparencyLoc`

`GLint` `udit::Object::transparencyLoc` [private]

Localización del uniform "transparency" en el shader.

La documentación de esta clase está generada de los siguientes archivos:

- [Object.hpp](#)
- [Object.cpp](#)

6.8. Referencia de la estructura

udit::Window::OpenGL_Context_Settings

Estructura para configurar los parámetros del contexto de OpenGL.

```
#include <Window.hpp>
```

Atributos públicos

- unsigned `version_major` = 3
- unsigned `version_minor` = 3
- bool `core_profile` = true
- unsigned `depth_buffer_size` = 24
- unsigned `stencil_buffer_size` = 0
- bool `enable_vsync` = true

6.8.1. Descripción detallada

Estructura para configurar los parámetros del contexto de OpenGL.

6.8.2. Documentación de datos miembro

6.8.2.1. `core_profile`

```
bool udit::Window::OpenGL_Context_Settings::core_profile = true
```

Indica si se debe usar el perfil core.

6.8.2.2. `depth_buffer_size`

```
unsigned udit::Window::OpenGL_Context_Settings::depth_buffer_size = 24
```

Tamaño del buffer de profundidad.

6.8.2.3. `enable_vsync`

```
bool udit::Window::OpenGL_Context_Settings::enable_vsync = true
```

Si se debe activar V-Sync.

6.8.2.4. `stencil_buffer_size`

```
unsigned udit::Window::OpenGL_Context_Settings::stencil_buffer_size = 0
```

Tamaño del buffer de stencil.

6.8.2.5. version__major

```
unsigned udit::Window::OpenGL_Context_Settings::version__major = 3
```

Versión mayor del contexto OpenGL.

6.8.2.6. version__minor

```
unsigned udit::Window::OpenGL_Context_Settings::version__minor = 3
```

Versión menor del contexto OpenGL.

La documentación de esta estructura está generada del siguiente archivo:

- [Window.hpp](#)

6.9. Referencia de la estructura Window::OpenGL_Context_Settings

Estructura para configurar los parámetros del contexto de OpenGL.

```
#include <Window.hpp>
```

Atributos públicos

- unsigned [version__major](#) = 3
- unsigned [version__minor](#) = 3
- bool [core__profile](#) = true
- unsigned [depth__buffer__size](#) = 24
- unsigned [stencil__buffer__size](#) = 0
- bool [enable__vsync](#) = true

6.9.1. Descripción detallada

Estructura para configurar los parámetros del contexto de OpenGL.

6.9.2. Documentación de datos miembro

6.9.2.1. core__profile

```
bool udit::Window::OpenGL_Context_Settings::core__profile = true
```

Indica si se debe usar el perfil core.

6.9.2.2. depth__buffer__size

```
unsigned udit::Window::OpenGL_Context_Settings::depth__buffer__size = 24
```

Tamaño del buffer de profundidad.

6.9.2.3. enable_vsync

```
bool udit::Window::OpenGL_Context_Settings::enable_vsync = true
```

Si se debe activar V-Sync.

6.9.2.4. stencil_buffer_size

```
unsigned udit::Window::OpenGL_Context_Settings::stencil_buffer_size = 0
```

Tamaño del buffer de stencil.

6.9.2.5. version_major

```
unsigned udit::Window::OpenGL_Context_Settings::version_major = 3
```

Versión mayor del contexto OpenGL.

6.9.2.6. version_minor

```
unsigned udit::Window::OpenGL_Context_Settings::version_minor = 3
```

Versión menor del contexto OpenGL.

La documentación de esta estructura está generada del siguiente archivo:

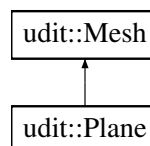
- [Window.hpp](#)

6.10. Referencia de la clase udit::Plane

Clase para representar un plano 3D.

```
#include <Plane.hpp>
```

Diagrama de herencia de udit::Plane



Métodos públicos

- [Plane](#) (int subdivisionsX, int subdivisionsY, float width, float height)
Constructor de la clase [Plane](#).
- [~Plane](#) ()
Destructor de la clase [Plane](#).

Métodos públicos heredados de `udit::Mesh`

- `Mesh ()`
Constructor por defecto. Inicializa la malla vacía y genera los buffers OpenGL.
- `Mesh (MeshData receivedData)`
Constructor con datos de malla.
- `void render ()`
Renderiza la malla usando los datos almacenados y buffers GPU.
- `void generateBuffers ()`
Genera los buffers OpenGL (VBOs y VAO) para la malla actual.
- `void uploadBuffer (GLuint bufferId, GLenum target, const void *dataPtr, size_t dataSize)`
Carga datos en un buffer de OpenGL.
- `void deleteBuffers ()`
Elimina los buffers OpenGL asociados a esta malla.

Métodos privados

- `void generateGeometry ()`
Genera la geometría del plano.

Atributos privados

- `float grid_width`
Número de columnas en la cuadrícula.
- `float grid_height`
Número de filas en la cuadrícula.
- `int subdivX`
- `int subdivY`

Otros miembros heredados

Tipos protegidos heredados de `udit::Mesh`

- `enum {
COORDINATES_VBO , TEXCOORDS_VBO , NORMALS_VBO , INDICES_EBO ,
VBO_COUNT }`
Índices para los diferentes buffers usados.

Atributos protegidos heredados de `udit::Mesh`

- `bool isInitialized`
Indica si los buffers están inicializados y listos para renderizar.
- `GLuint vao_id`
Identificador del Vertex Array Object.
- `GLuint vbo_ids [VBO_COUNT]`
Identificadores de Vertex Buffer Objects y Element Buffer Object.
- `MeshData data`
Datos de la malla: coordenadas, normales, texturas e índices.

6.10.1. Descripción detallada

Clase para representar un plano 3D.

Esta clase genera un plano de malla utilizando OpenGL. El plano tiene una cuadrícula definida por su ancho y alto, y se utiliza para representar superficies planas.

6.10.2. Documentación de constructores y destructores

6.10.2.1. Plane()

```
udit::Plane::Plane (
    int subdivisionsX,
    int subdivisionsY,
    float width,
    float height)
```

Constructor de la clase [Plane](#).

Inicializa un plano con la cuadrícula definida por el ancho y alto proporcionados. Genera la geometría del plano (vértices, coordenadas de textura e índices).

Parámetros

width	El ancho del plano, define la cantidad de columnas de la cuadrícula.
height	La altura del plano, define la cantidad de filas de la cuadrícula.

6.10.2.2. ~Plane()

```
udit::Plane::~Plane ()
```

Destructor de la clase [Plane](#).

Libera los recursos de OpenGL utilizados para almacenar la geometría del plano.

6.10.3. Documentación de funciones miembro

6.10.3.1. generateGeometry()

```
void udit::Plane::generateGeometry () [private]
```

Genera la geometría del plano.

Calcula las posiciones de los vértices, las coordenadas de textura y los índices que definen la malla del plano en función de las dimensiones de la cuadrícula.

6.10.4. Documentación de datos miembro

6.10.4.1. grid_height

float udit::Plane::grid_height [private]

N mero de filas en la cuadr cula.

6.10.4.2. grid_width

float udit::Plane::grid_width [private]

N mero de columnas en la cuadr cula.

6.10.4.3. subdivX

int udit::Plane::subdivX [private]

6.10.4.4. subdivY

int udit::Plane::subdivY [private]

La documentación de esta clase está generada de los siguientes archivos:

- [Plane.hpp](#)
- [Plane.cpp](#)

6.11. Referencia de la clase Scene

Representa una escena 3D con cámara, objetos, skybox, texturas, iluminación y shaders.

```
#include <Scene.hpp>
```

Métodos públicos

- [Scene](#) (unsigned width, unsigned height)
Constructor de [Scene](#).
- [~Scene](#) ()
- void [loadTextures](#) ()
Carga las texturas requeridas desde disco y las transfiere a la GPU.
- void [setTextures](#) ()
Asigna los identificadores de textura a los objetos 3D correspondientes.
- void [update](#) ()
Actualiza parámetros dinámicos de la escena (como rotaciones).
- void [render](#) ()
Renderiza toda la escena, incluyendo skybox, terreno y objetos.
- void [applyLightSettings](#) (ShaderProgram &shader, const vector< glm::vec3 > &dirsView, const vector< glm::vec3 > &colors, const vector< float > &intensities)
- void [lightSetup](#) (glm::mat4 &view_matrix)
Configura las fuentes de luz y sus propiedades en los shaders activos.
- void [resize](#) (unsigned width, unsigned height)
Recalcula la matriz de proyección y ajusta el viewport tras un redimensionado.
- void [set_camera](#) (Camera new_camera)
Establece una nueva instancia de cámara para la escena.

Métodos privados

- void [initResources](#) ()
- void [initObjects](#) ()
- void [initSceneGraph](#) ()

Atributos privados

- GeometryGenerator [generator](#)
Generador de primitivas geométricas.
- unique_ptr< ShaderProgram > [defaultProgram](#)
Shader con iluminación estándar.
- unique_ptr< ShaderProgram > [unlitProgram](#)
Shader sin iluminación (para objetos como el cono).
- unique_ptr< ShaderProgram > [skyboxProgram](#)
Shader para renderizar el skybox.
- unique_ptr< ShaderProgram > [heightmapProgram](#)
Shader especializado en renderizar heightmaps.
- glm::vec3 [lightPos](#)
Posición de la luz principal (no usada directamente si se usan direcciones).
- glm::vec3 [lightColor](#)
Color de la luz.
- glm::vec3 [viewPos](#)
Posición de la cámara en el espacio de mundo.
- glm::mat4 [projection_matrix](#)
Matriz de proyección en perspectiva.
- unique_ptr< AssimpMesh > [ufo](#)
Modelo 3D del OVNI.
- unique_ptr< AssimpMesh > [cow](#)
Modelo 3D de la vaca.
- unique_ptr< Mesh > [plane](#)
Malla del plano (heightmap).
- unique_ptr< Mesh > [cone](#)
Malla del cono.
- unique_ptr< Object > [heightmapObj](#)
Objeto gráfico del heightmap.
- unique_ptr< Object > [ufoObj](#)
Objeto gráfico del OVNI.
- unique_ptr< Object > [cowObj](#)
Objeto gráfico de la vaca.
- unique_ptr< Object > [coneObj](#)
Objeto gráfico del cono.
- unique_ptr< Skybox > [skybox](#)
Objeto que representa el skybox de fondo.
- float [angle](#)
Ángulo de rotación usado para animación.
- Camera [camera](#)
Cámara activa de la escena.
- TextureLoader [textureLoader](#)
Gestor de carga de texturas.
- GLuint [coneTextureID](#)

- ID de la textura del cono.
- GLuint [skyboxTextureID](#)
 - ID del cubemap del skybox.
- GLuint [heightmapID](#)
 - ID de la textura de altura.
- GLuint [heightmapTextureID](#)
 - ID de la textura difusa para el heightmap.
- GLuint [cowTextureID](#)
 - ID de la textura de la vaca.
- GLuint [ufoTextureID](#)
 - ID de la textura del OVNI.
- std::shared_ptr< SceneNode > [rootNode](#)
 - Nodo raíz de la jerarquía de escena.
- std::shared_ptr< SceneNode > [ufoCowConeNode](#)
 - Nodo contenedor de OVNI, vaca y cono.
- std::shared_ptr< SceneNode > [ufoNode](#)
 - Nodo del OVNI.
- std::shared_ptr< SceneNode > [cowNode](#)
 - Nodo de la vaca.
- std::shared_ptr< SceneNode > [coneNode](#)
 - Nodo del cono.
- std::shared_ptr< SceneNode > [heightmapNode](#)
 - Nodo del terreno con heightmap.

Atributos estáticos privados

- static const float [UFO_HEIGHT](#) = 250.f
 - Altura base del modelo OVNI.
- static const float [COW_HEIGHT](#) = -100.f
 - Altura base del modelo vaca.
- static const float [CONE_HEIGHT](#) = -400.f
 - Altura base del modelo cono.
- static const float [SCALE_SMALL](#) = 0.1f
 - Escala reducida para modelos pequeños.
- static const float [SCALE_BIG](#) = 10.f
 - Escala ampliada para modelos grandes.

6.11.1. Descripción detallada

Representa una escena 3D con cámara, objetos, skybox, texturas, iluminación y shaders.

Esta clase se encarga de gestionar la creación, configuración, actualización y renderizado de una escena 3D compuesta por distintos objetos jerárquicos. También configura el sistema de iluminación y controla la cámara.

6.11.2. Documentación de constructores y destructores

6.11.2.1. Scene()

```
udit::Scene::Scene (
    unsigned width,
    unsigned height)
```

Constructor de [Scene](#).

Parámetros

width	Ancho de la ventana de visualización.
height	Alto de la ventana de visualización.

Inicializa la cámara, objetos, shaders, texturas y configura la jerarquía de escena.

6.11.2.2. `~Scene()`

`udit::Scene::~Scene ()` [default]

6.11.3. Documentación de funciones miembro

6.11.3.1. `applyLightSettings()`

```
void udit::Scene::applyLightSettings (
    ShaderProgram & shader,
    const vector< glm::vec3 > & dirsView,
    const vector< glm::vec3 > & colors,
    const vector< float > & intensities)
```

6.11.3.2. `initObjects()`

`void udit::Scene::initObjects ()` [private]

6.11.3.3. `initResources()`

`void udit::Scene::initResources ()` [private]

6.11.3.4. `initSceneGraph()`

`void udit::Scene::initSceneGraph ()` [private]

6.11.3.5. `lightSetup()`

```
void udit::Scene::lightSetup (
    glm::mat4 & view_matrix)
```

Configura las fuentes de luz y sus propiedades en los shaders activos.

Parámetros

view_matrix	Matriz de vista desde la cámara.
-------------	----------------------------------

6.11.3.6. loadTextures()

```
void udit::Scene::loadTextures ()
```

Carga las texturas requeridas desde disco y las transfiere a la GPU.

6.11.3.7. render()

```
void udit::Scene::render ()
```

Renderiza toda la escena, incluyendo skybox, terreno y objetos.

6.11.3.8. resize()

```
void udit::Scene::resize (  
    unsigned width,  
    unsigned height)
```

Recalcula la matriz de proyección y ajusta el viewport tras un redimensionado.

Parámetros

width	Nuevo ancho de la ventana.
height	Nuevo alto de la ventana.

6.11.3.9. set_camera()

```
void udit::Scene::set_camera (  
    Camera new_camera)
```

Establece una nueva instancia de cámara para la escena.

Parámetros

new_camera	Objeto Camera que será usado para generar la vista.
------------	---

6.11.3.10. setTextures()

```
void udit::Scene::setTextures ()
```

Asigna los identificadores de textura a los objetos 3D correspondientes.

6.11.3.11. update()

```
void udit::Scene::update ()
```

Actualiza parámetros dinámicos de la escena (como rotaciones).

6.11.4. Documentación de datos miembro

6.11.4.1. angle

`float udit::Scene::angle [private]`

Ángulo de rotación usado para animación.

6.11.4.2. camera

`Camera udit::Scene::camera [private]`

Cámara activa de la escena.

6.11.4.3. cone

`unique_ptr<Mesh> udit::Scene::cone [private]`

Malla del cono.

6.11.4.4. CONE_HEIGHT

`float udit::Scene::CONE_HEIGHT = -400.f [static], [constexpr], [private]`

Altura base del modelo cono.

6.11.4.5. coneNode

`std::shared_ptr<SceneNode> udit::Scene::coneNode [private]`

Nodo del cono.

6.11.4.6. coneObj

`unique_ptr<Object> udit::Scene::coneObj [private]`

Objeto gráfico del cono.

6.11.4.7. coneTextureID

`GLuint udit::Scene::coneTextureID [private]`

ID de la textura del cono.

6.11.4.8. cow

```
unique_ptr<AssimpMesh> udit::Scene::cow [private]
```

Modelo 3D de la vaca.

6.11.4.9. COW_HEIGHT

```
float udit::Scene::COW_HEIGHT = -100.f [static], [constexpr], [private]
```

Altura base del modelo vaca.

6.11.4.10. cowNode

```
std::shared_ptr<SceneNode> udit::Scene::cowNode [private]
```

Nodo de la vaca.

6.11.4.11. cowObj

```
unique_ptr<Object> udit::Scene::cowObj [private]
```

Objeto gráfico de la vaca.

6.11.4.12. cowTextureID

```
GLuint udit::Scene::cowTextureID [private]
```

ID de la textura de la vaca.

6.11.4.13. defaultProgram

```
unique_ptr<ShaderProgram> udit::Scene::defaultProgram [private]
```

Shader con iluminación estándar.

6.11.4.14. generator

```
GeometryGenerator udit::Scene::generator [private]
```

Generador de primitivas geométricas.

6.11.4.15. heightmapID

```
GLuint udit::Scene::heightmapID [private]
```

ID de la textura de altura.

6.11.4.16. heightmapNode

`std::shared_ptr<SceneNode> udit::Scene::heightmapNode [private]`

Nodo del terreno con heightmap.

6.11.4.17. heightmapObj

`unique_ptr<Object> udit::Scene::heightmapObj [private]`

Objeto gráfico del heightmap.

6.11.4.18. heightmapProgram

`unique_ptr<ShaderProgram> udit::Scene::heightmapProgram [private]`

Shader especializado en renderizar heightmaps.

6.11.4.19. heightmapTextureID

`GLuint udit::Scene::heightmapTextureID [private]`

ID de la textura difusa para el heightmap.

6.11.4.20. lightColor

`glm::vec3 udit::Scene::lightColor [private]`

Color de la luz.

6.11.4.21. lightPos

`glm::vec3 udit::Scene::lightPos [private]`

Posición de la luz principal (no usada directamente si se usan direcciones).

6.11.4.22. plane

`unique_ptr<Mesh> udit::Scene::plane [private]`

Malla del plano (heightmap).

6.11.4.23. projection_matrix

`glm::mat4 udit::Scene::projection_matrix [private]`

Matriz de proyección en perspectiva.

6.11.4.24. rootNode

```
std::shared_ptr<SceneNode> udit::Scene::rootNode [private]
```

Nodo raíz de la jerarquía de escena.

6.11.4.25. SCALE_BIG

```
float udit::Scene::SCALE_BIG = 10.f [static], [constexpr], [private]
```

Escala ampliada para modelos grandes.

6.11.4.26. SCALE_SMALL

```
float udit::Scene::SCALE_SMALL = 0.1f [static], [constexpr], [private]
```

Escala reducida para modelos pequeños.

6.11.4.27. skybox

```
unique_ptr<Skybox> udit::Scene::skybox [private]
```

Objeto que representa el skybox de fondo.

6.11.4.28. skyboxProgram

```
unique_ptr<ShaderProgram> udit::Scene::skyboxProgram [private]
```

Shader para renderizar el skybox.

6.11.4.29. skyboxTextureID

```
GLuint udit::Scene::skyboxTextureID [private]
```

ID del cubemap del skybox.

6.11.4.30. textureLoader

```
TextureLoader udit::Scene::textureLoader [private]
```

Gestor de carga de texturas.

6.11.4.31. ufo

```
unique_ptr<AssimpMesh> udit::Scene::ufo [private]
```

Modelo 3D del OVNI.

6.11.4.32. UFO_HEIGHT

```
float udit::Scene::UFO_HEIGHT = 250.f [static], [constexpr], [private]
```

Altura base del modelo OVNI.

6.11.4.33. ufoCowConeNode

```
std::shared_ptr<SceneNode> udit::Scene::ufoCowConeNode [private]
```

Nodo contenedor de OVNI, vaca y cono.

6.11.4.34. ufoNode

```
std::shared_ptr<SceneNode> udit::Scene::ufoNode [private]
```

Nodo del OVNI.

6.11.4.35. ufoObj

```
unique_ptr<Object> udit::Scene::ufoObj [private]
```

Objeto gráfico del OVNI.

6.11.4.36. ufoTextureID

```
GLuint udit::Scene::ufoTextureID [private]
```

ID de la textura del OVNI.

6.11.4.37. unlitProgram

```
unique_ptr<ShaderProgram> udit::Scene::unlitProgram [private]
```

Shader sin iluminación (para objetos como el cono).

6.11.4.38. viewPos

```
glm::vec3 udit::Scene::viewPos [private]
```

Posición de la cámara en el espacio de mundo.

La documentación de esta clase está generada de los siguientes archivos:

- [Scene.hpp](#)
- [Scene.cpp](#)

6.12. Referencia de la clase udit::Scene

Representa una escena 3D con cámara, objetos, skybox, texturas, iluminación y shaders.

```
#include <Scene.hpp>
```

Métodos públicos

- [Scene](#) (unsigned width, unsigned height)
Constructor de [Scene](#).
- [~Scene](#) ()
- void [loadTextures](#) ()
Carga las texturas requeridas desde disco y las transfiere a la GPU.
- void [setTextures](#) ()
Asigna los identificadores de textura a los objetos 3D correspondientes.
- void [update](#) ()
Actualiza parámetros dinámicos de la escena (como rotaciones).
- void [render](#) ()
Renderiza toda la escena, incluyendo skybox, terreno y objetos.
- void [applyLightSettings](#) ([ShaderProgram](#) &shader, const vector< glm::vec3 > &dirsView, const vector< glm::vec3 > &colors, const vector< float > &intensities)
- void [lightSetup](#) (glm::mat4 &view_matrix)
Configura las fuentes de luz y sus propiedades en los shaders activos.
- void [resize](#) (unsigned width, unsigned height)
Recalcula la matriz de proyección y ajusta el viewport tras un redimensionado.
- void [set_camera](#) ([Camera](#) new_camera)
Establece una nueva instancia de cámara para la escena.

Métodos privados

- void [initResources](#) ()
- void [initObjects](#) ()
- void [initSceneGraph](#) ()

Atributos privados

- [GeometryGenerator](#) generator
Generador de primitivas geométricas.
- unique_ptr< [ShaderProgram](#) > [defaultProgram](#)
Shader con iluminación estándar.
- unique_ptr< [ShaderProgram](#) > [unlitProgram](#)
Shader sin iluminación (para objetos como el cono).
- unique_ptr< [ShaderProgram](#) > [skyboxProgram](#)
Shader para renderizar el skybox.
- unique_ptr< [ShaderProgram](#) > [heightmapProgram](#)
Shader especializado en renderizar heightmaps.
- glm::vec3 [lightPos](#)
Posición de la luz principal (no usada directamente si se usan direcciones).
- glm::vec3 [lightColor](#)
Color de la luz.

- glm::vec3 [viewPos](#)
Posición de la cámara en el espacio de mundo.
- glm::mat4 [projection_matrix](#)
Matriz de proyección en perspectiva.
- unique_ptr< [AssimpMesh](#) > [ufo](#)
Modelo 3D del OVNI.
- unique_ptr< [AssimpMesh](#) > [cow](#)
Modelo 3D de la vaca.
- unique_ptr< [Mesh](#) > [plane](#)
Malla del plano (heightmap).
- unique_ptr< [Mesh](#) > [cone](#)
Malla del cono.
- unique_ptr< [Object](#) > [heightmapObj](#)
Objeto gráfico del heightmap.
- unique_ptr< [Object](#) > [ufoObj](#)
Objeto gráfico del OVNI.
- unique_ptr< [Object](#) > [cowObj](#)
Objeto gráfico de la vaca.
- unique_ptr< [Object](#) > [coneObj](#)
Objeto gráfico del cono.
- unique_ptr< [Skybox](#) > [skybox](#)
Objeto que representa el skybox de fondo.
- float [angle](#)
Ángulo de rotación usado para animación.
- [Camera](#) [camera](#)
Cámara activa de la escena.
- [TextureLoader](#) [textureLoader](#)
Gestor de carga de texturas.
- GLuint [coneTextureID](#)
ID de la textura del cono.
- GLuint [skyboxTextureID](#)
ID del cubemap del skybox.
- GLuint [heightmapID](#)
ID de la textura de altura.
- GLuint [heightmapTextureID](#)
ID de la textura difusa para el heightmap.
- GLuint [cowTextureID](#)
ID de la textura de la vaca.
- GLuint [ufoTextureID](#)
ID de la textura del OVNI.
- std::shared_ptr< [SceneNode](#) > [rootNode](#)
Nodo raíz de la jerarquía de escena.
- std::shared_ptr< [SceneNode](#) > [ufoCowConeNode](#)
Nodo contenedor de OVNI, vaca y cono.
- std::shared_ptr< [SceneNode](#) > [ufoNode](#)
Nodo del OVNI.
- std::shared_ptr< [SceneNode](#) > [cowNode](#)
Nodo de la vaca.
- std::shared_ptr< [SceneNode](#) > [coneNode](#)
Nodo del cono.
- std::shared_ptr< [SceneNode](#) > [heightmapNode](#)
Nodo del terreno con heightmap.

Atributos estáticos privados

- static const float [UFO_HEIGHT](#) = 250.f
Altura base del modelo OVNI.
- static const float [COW_HEIGHT](#) = -100.f
Altura base del modelo vaca.
- static const float [CONE_HEIGHT](#) = -400.f
Altura base del modelo cono.
- static const float [SCALE_SMALL](#) = 0.1f
Escala reducida para modelos pequeños.
- static const float [SCALE_BIG](#) = 10.f
Escala ampliada para modelos grandes.

6.12.1. Descripción detallada

Representa una escena 3D con cámara, objetos, skybox, texturas, iluminación y shaders.

Esta clase se encarga de gestionar la creación, configuración, actualización y renderizado de una escena 3D compuesta por distintos objetos jerárquicos. También configura el sistema de iluminación y controla la cámara.

6.12.2. Documentación de constructores y destructores

6.12.2.1. Scene()

```
udit::Scene::Scene (
    unsigned width,
    unsigned height)
```

Constructor de [Scene](#).

Parámetros

width	Ancho de la ventana de visualización.
height	Alto de la ventana de visualización.

Inicializa la cámara, objetos, shaders, texturas y configura la jerarquía de escena.

6.12.2.2. ~Scene()

```
udit::Scene::~Scene () [default]
```

6.12.3. Documentación de funciones miembro

6.12.3.1. applyLightSettings()

```
void udit::Scene::applyLightSettings (
    ShaderProgram & shader,
    const vector< glm::vec3 > & dirsView,
    const vector< glm::vec3 > & colors,
    const vector< float > & intensities)
```

6.12.3.2. initObjects()

```
void udit::Scene::initObjects () [private]
```

6.12.3.3. initResources()

```
void udit::Scene::initResources () [private]
```

6.12.3.4. initSceneGraph()

```
void udit::Scene::initSceneGraph () [private]
```

6.12.3.5. lightSetup()

```
void udit::Scene::lightSetup (
    glm::mat4 & view_matrix)
```

Configura las fuentes de luz y sus propiedades en los shaders activos.

Parámetros

view_matrix	Matriz de vista desde la cámara.
-------------	----------------------------------

6.12.3.6. loadTextures()

```
void udit::Scene::loadTextures ()
```

Carga las texturas requeridas desde disco y las transfiere a la GPU.

6.12.3.7. render()

```
void udit::Scene::render ()
```

Renderiza toda la escena, incluyendo skybox, terreno y objetos.

6.12.3.8. resize()

```
void udit::Scene::resize (
    unsigned width,
    unsigned height)
```

Recalcula la matriz de proyección y ajusta el viewport tras un redimensionado.

Parámetros

width	Nuevo ancho de la ventana.
height	Nuevo alto de la ventana.

6.12.3.9. set_camera()

```
void udit::Scene::set_camera (
    Camera new_camera)
```

Establece una nueva instancia de cámara para la escena.

Parámetros

new_camera	Objeto Camera que será usado para generar la vista.
------------	---

6.12.3.10. setTextures()

```
void udit::Scene::setTextures ()
```

Asigna los identificadores de textura a los objetos 3D correspondientes.

6.12.3.11. update()

```
void udit::Scene::update ()
```

Actualiza parámetros dinámicos de la escena (como rotaciones).

6.12.4. Documentación de datos miembro

6.12.4.1. angle

```
float udit::Scene::angle [private]
```

Ángulo de rotación usado para animación.

6.12.4.2. camera

```
Camera udit::Scene::camera [private]
```

Cámara activa de la escena.

6.12.4.3. cone

```
unique_ptr<Mesh> udit::Scene::cone [private]
```

Malla del cono.

6.12.4.4. CONE_HEIGHT

```
float udit::Scene::CONE_HEIGHT = -400.f [static], [constexpr], [private]
```

Altura base del modelo cono.

6.12.4.5. coneNode

```
std::shared_ptr<SceneNode> udit::Scene::coneNode [private]
```

Nodo del cono.

6.12.4.6. coneObj

`unique_ptr<Object> udit::Scene::coneObj [private]`

Objeto gráfico del cono.

6.12.4.7. coneTextureID

`GLuint udit::Scene::coneTextureID [private]`

ID de la textura del cono.

6.12.4.8. cow

`unique_ptr<AssimpMesh> udit::Scene::cow [private]`

Modelo 3D de la vaca.

6.12.4.9. COW_HEIGHT

`float udit::Scene::COW_HEIGHT = -100.f [static], [constexpr], [private]`

Altura base del modelo vaca.

6.12.4.10. cowNode

`std::shared_ptr<SceneNode> udit::Scene::cowNode [private]`

Nodo de la vaca.

6.12.4.11. cowObj

`unique_ptr<Object> udit::Scene::cowObj [private]`

Objeto gráfico de la vaca.

6.12.4.12. cowTextureID

`GLuint udit::Scene::cowTextureID [private]`

ID de la textura de la vaca.

6.12.4.13. defaultProgram

`unique_ptr<ShaderProgram> udit::Scene::defaultProgram [private]`

Shader con iluminación estándar.

6.12.4.14. generator

[GeometryGenerator](#) udit::Scene::generator [private]

Generador de primitivas geométricas.

6.12.4.15. heightmapID

GLuint udit::Scene::heightmapID [private]

ID de la textura de altura.

6.12.4.16. heightmapNode

std::shared_ptr<[SceneNode](#)> udit::Scene::heightmapNode [private]

Nodo del terreno con heightmap.

6.12.4.17. heightmapObj

unique_ptr<[Object](#)> udit::Scene::heightmapObj [private]

Objeto gráfico del heightmap.

6.12.4.18. heightmapProgram

unique_ptr<[ShaderProgram](#)> udit::Scene::heightmapProgram [private]

Shader especializado en renderizar heightmaps.

6.12.4.19. heightmapTextureID

GLuint udit::Scene::heightmapTextureID [private]

ID de la textura difusa para el heightmap.

6.12.4.20. lightColor

glm::vec3 udit::Scene::lightColor [private]

Color de la luz.

6.12.4.21. lightPos

glm::vec3 udit::Scene::lightPos [private]

Posición de la luz principal (no usada directamente si se usan direcciones).

6.12.4.22. plane

```
unique_ptr<Mesh> udit::Scene::plane [private]
```

Malla del plano (heightmap).

6.12.4.23. projection_matrix

```
glm::mat4 udit::Scene::projection_matrix [private]
```

Matriz de proyección en perspectiva.

6.12.4.24. rootNode

```
std::shared_ptr<SceneNode> udit::Scene::rootNode [private]
```

Nodo raíz de la jerarquía de escena.

6.12.4.25. SCALE_BIG

```
float udit::Scene::SCALE_BIG = 10.f [static], [constexpr], [private]
```

Escala ampliada para modelos grandes.

6.12.4.26. SCALE_SMALL

```
float udit::Scene::SCALE_SMALL = 0.1f [static], [constexpr], [private]
```

Escala reducida para modelos pequeños.

6.12.4.27. skybox

```
unique_ptr<Skybox> udit::Scene::skybox [private]
```

Objeto que representa el skybox de fondo.

6.12.4.28. skyboxProgram

```
unique_ptr<ShaderProgram> udit::Scene::skyboxProgram [private]
```

Shader para renderizar el skybox.

6.12.4.29. skyboxTextureID

```
GLuint udit::Scene::skyboxTextureID [private]
```

ID del cubemap del skybox.

6.12.4.30. textureLoader

`TextureLoader` udit::Scene::textureLoader [private]

Gestor de carga de texturas.

6.12.4.31. ufo

`unique_ptr<AssimpMesh>` udit::Scene::ufo [private]

Modelo 3D del OVNI.

6.12.4.32. UFO_HEIGHT

`float` udit::Scene::UFO_HEIGHT = 250.f [static], [constexpr], [private]

Altura base del modelo OVNI.

6.12.4.33. ufoCowConeNode

`std::shared_ptr<SceneNode>` udit::Scene::ufoCowConeNode [private]

Nodo contenedor de OVNI, vaca y cono.

6.12.4.34. ufoNode

`std::shared_ptr<SceneNode>` udit::Scene::ufoNode [private]

Nodo del OVNI.

6.12.4.35. ufoObj

`unique_ptr<Object>` udit::Scene::ufoObj [private]

Objeto gráfico del OVNI.

6.12.4.36. ufoTextureID

`GLuint` udit::Scene::ufoTextureID [private]

ID de la textura del OVNI.

6.12.4.37. unlitProgram

`unique_ptr<ShaderProgram>` udit::Scene::unlitProgram [private]

Shader sin iluminación (para objetos como el cono).

6.12.4.38. viewPos

glm::vec3 udit::Scene::viewPos [private]

Posición de la cámara en el espacio de mundo.

La documentación de esta clase está generada de los siguientes archivos:

- [Scene.hpp](#)
- [Scene.cpp](#)

6.13. Referencia de la clase udit::SceneNode

Nodo de escena para jerarquía de objetos 3D.

```
#include <SceneNode.hpp>
```

Métodos públicos

- [SceneNode](#) ([Object](#) *[object](#)=nullptr)
Constructor.
- void [addChild](#) (std::shared_ptr< [SceneNode](#) > child)
Añade un nodo hijo a este nodo.
- void [setTransform](#) (const glm::vec3 &[position](#), const glm::vec3 &[rotation](#), const glm::vec3 &[scale](#))
Define la transformación local del nodo.
- void [render](#) (const glm::mat4 &parentTransform, const glm::mat4 &view, const glm::mat4 &projection)
Renderiza el nodo y sus hijos.

Métodos privados

- glm::mat4 [computeLocalTransform](#) () const
Calcula la matriz de transformación local (modelo).

Atributos privados

- [Object](#) * [object](#)
Objeto representado en este nodo (puede ser nullptr).
- glm::vec3 [position](#)
Posición local.
- glm::vec3 [rotation](#)
Rotación local (grados).
- glm::vec3 [scale](#)
Escala local.
- std::vector< std::shared_ptr< [SceneNode](#) > > [children](#)
Nodos hijos.

6.13.1. Descripción detallada

Nodo de escena para jerarquía de objetos 3D.

Representa un nodo en un grafo de escena, que puede contener un objeto y varios nodos hijos, con transformaciones locales acumulables.

6.13.2. Documentación de constructores y destructores

6.13.2.1. SceneNode()

```
udit::SceneNode::SceneNode (  
    Object * object = nullptr)
```

Constructor.

Parámetros

object	Puntero a un objeto que este nodo representa. Puede ser nullptr.
--------	--

6.13.3. Documentación de funciones miembro

6.13.3.1. addChild()

```
void udit::SceneNode::addChild (  
    std::shared_ptr< SceneNode > child)
```

Añade un nodo hijo a este nodo.

Parámetros

child	Nodo hijo a añadir.
-------	---------------------

6.13.3.2. computeLocalTransform()

```
glm::mat4 udit::SceneNode::computeLocalTransform () const [private]
```

Calcula la matriz de transformación local (modelo).

Devuelve

Matriz 4x4 que representa la transformación local.

6.13.3.3. render()

```
void udit::SceneNode::render (  
    const glm::mat4 & parentTransform,  
    const glm::mat4 & view,  
    const glm::mat4 & projection)
```

Renderiza el nodo y sus hijos.

Parámetros

parentTransform	Matriz de transformación acumulada del nodo padre.
view	Matriz de vista de la cámara.
projection	Matriz de proyección.

Se calcula la transformación global y se pasa a los objetos para renderizar.

6.13.3.4. setTransform()

```
void udit::SceneNode::setTransform (
    const glm::vec3 & position,
    const glm::vec3 & rotation,
    const glm::vec3 & scale)
```

Define la transformación local del nodo.

Parámetros

position	Vector de posición local.
rotation	Vector de rotación local en grados (euler angles).
scale	Vector de escala local.

6.13.4. Documentación de datos miembro

6.13.4.1. children

```
std::vector<std::shared_ptr<SceneNode> > udit::SceneNode::children [private]
```

Nodos hijos.

6.13.4.2. object

```
Object* udit::SceneNode::object [private]
```

Objeto representado en este nodo (puede ser nullptr).

6.13.4.3. position

```
glm::vec3 udit::SceneNode::position [private]
```

Posición local.

6.13.4.4. rotation

```
glm::vec3 udit::SceneNode::rotation [private]
```

Rotación local (grados).

6.13.4.5. scale

```
glm::vec3 udit::SceneNode::scale [private]
```

Escala local.

La documentación de esta clase está generada de los siguientes archivos:

- [SceneNode.hpp](#)
- [SceneNode.cpp](#)

6.14. Referencia de la clase udit::ShaderProgram

Clase para manejar programas de sombreado (shaders) en OpenGL.

```
#include <ShaderProgram.hpp>
```

Métodos públicos

- [ShaderProgram](#) (const std::string &vertex_path, const std::string &fragment_path)
Constructor que compila y enlaza los shaders.
- [~ShaderProgram](#) ()
Destructor. Libera el programa de OpenGL.
- [ShaderProgram](#) (const [ShaderProgram](#) &)=delete
- [ShaderProgram](#) & operator= (const [ShaderProgram](#) &)=delete
- [ShaderProgram](#) ([ShaderProgram](#) &&other) noexcept
Constructor de movimiento.
- [ShaderProgram](#) & operator= ([ShaderProgram](#) &&other) noexcept
Operador de asignación por movimiento.
- void [use](#) () const
Activa el programa para su uso en el pipeline de OpenGL.
- GLuint [id](#) () const
Obtiene el ID del programa en OpenGL.
- void [setMat4](#) (const std::string &name, const glm::mat4 &mat) const
Sube una matriz 4x4 a un uniform del shader.
- void [setVec3](#) (const std::string &name, const glm::vec3 &vec) const
Sube un vector 3D a un uniform del shader.
- void [setFloat](#) (const std::string &name, float value) const
Sube un valor float a un uniform del shader.
- void [setInt](#) (const std::string &name, int value) const
Sube un valor entero a un uniform del shader.

Métodos privados

- void [show_compilation_error](#) (GLuint shader_id) const
Muestra un error de compilación de un shader.
- void [show_linkage_error](#) (GLuint program_id) const
Muestra un error de enlace del programa.
- GLint [getUniformLocation](#) (const std::string &name) const
Obtiene la ubicación de un uniform en el programa, cacheándola para futuros accesos.

Atributos privados

- GLuint [program_id](#)
- std::unordered_map< std::string, GLint > [uniform_locations](#)

6.14.1. Descripción detallada

Clase para manejar programas de sombreado (shaders) en OpenGL.

Permite compilar, enlazar y usar shaders de vértices y fragmentos, así como subir uniforms.

6.14.2. Documentación de constructores y destructores

6.14.2.1. ShaderProgram() [1/3]

```
udit::ShaderProgram::ShaderProgram (
    const std::string & vertex__path,
    const std::string & fragment__path)
```

Constructor que compila y enlaza los shaders.

Parámetros

vertex_source	Código fuente del shader de vértices.
fragment_source	Código fuente del shader de fragmentos.

6.14.2.2. ~ShaderProgram()

```
udit::ShaderProgram::~~ShaderProgram ()
```

Destructor. Libera el programa de OpenGL.

6.14.2.3. ShaderProgram() [2/3]

```
udit::ShaderProgram::ShaderProgram (
    const ShaderProgram & ) [delete]
```

No se permite copiar el programa.

6.14.2.4. ShaderProgram() [3/3]

```
udit::ShaderProgram::ShaderProgram (
    ShaderProgram && other) [noexcept]
```

Constructor de movimiento.

Parámetros

other	Objeto ShaderProgram que será movido.
-------	---

6.14.3. Documentación de funciones miembro

6.14.3.1. glGetUniformLocation()

```
GLint udit::ShaderProgram::getUniformLocation (
    const std::string & name) const [private]
```

Obtiene la ubicación de un uniform en el programa, cacheándola para futuros accesos.

Parámetros

name	Nombre del uniform.
------	---------------------

Devuelve

Ubicación del uniform, o -1 si no existe.

6.14.3.2. id()

```
GLuint udit::ShaderProgram::id () const [inline]
```

Obtiene el ID del programa en OpenGL.

Devuelve

ID del programa.

6.14.3.3. operator=() [1/2]

```
ShaderProgram & udit::ShaderProgram::operator= (
    const ShaderProgram & ) [delete]
```

No se permite asignar por copia.

6.14.3.4. operator=() [2/2]

```
ShaderProgram & udit::ShaderProgram::operator= (
    ShaderProgram && other) [noexcept]
```

Operador de asignación por movimiento.

Parámetros

other	Objeto ShaderProgram que será movido.
-------	---

Devuelve

Referencia al objeto actual.

6.14.3.5. setFloat()

```
void udit::ShaderProgram::setFloat (  
    const std::string & name,  
    float value) const
```

Sube un valor float a un uniform del shader.

Parámetros

name	Nombre del uniform.
value	Valor flotante a subir.

6.14.3.6. setInt()

```
void udit::ShaderProgram::setInt (  
    const std::string & name,  
    int value) const
```

Sube un valor entero a un uniform del shader.

Parámetros

name	Nombre del uniform.
value	Valor entero a subir.

6.14.3.7. setMat4()

```
void udit::ShaderProgram::setMat4 (  
    const std::string & name,  
    const glm::mat4 & mat) const
```

Sube una matriz 4x4 a un uniform del shader.

Parámetros

name	Nombre del uniform.
mat	Matriz a subir.

6.14.3.8. setVec3()

```
void udit::ShaderProgram::setVec3 (  
    const std::string & name,  
    const glm::vec3 & vec) const
```

Sube un vector 3D a un uniform del shader.

Parámetros

name	Nombre del uniform.
vec	Vector a subir.

6.14.3.9. show_compilation_error()

```
void udit::ShaderProgram::show_compilation_error (  
    GLuint shader_id) const    [private]
```

Muestra un error de compilación de un shader.

Parámetros

shader_id	ID del shader que falló al compilar.
-----------	--------------------------------------

6.14.3.10. show_linkage_error()

```
void udit::ShaderProgram::show_linkage_error (  
    GLuint program_id) const    [private]
```

Muestra un error de enlace del programa.

Parámetros

program_id	ID del programa que falló al enlazar.
------------	---------------------------------------

6.14.3.11. use()

```
void udit::ShaderProgram::use () const
```

Activa el programa para su uso en el pipeline de OpenGL.

6.14.4. Documentación de datos miembro

6.14.4.1. program_id

```
GLuint udit::ShaderProgram::program_id    [private]
```

Identificador del programa de shaders en OpenGL.

6.14.4.2. uniform_locations

`std::unordered_map<std::string, GLint> udit::ShaderProgram::uniform_locations [mutable], [private]`

Cache de ubicaciones de uniforms para optimizar llamadas.

La documentación de esta clase está generada de los siguientes archivos:

- [ShaderProgram.hpp](#)
- [ShaderProgram.cpp](#)

6.15. Referencia de la clase `udit::Skybox`

Clase que representa un skybox para el entorno 3D.

```
#include <Skybox.hpp>
```

Métodos públicos

- [Skybox](#) ([MeshData](#) mesh, [ShaderProgram](#) *shader)
Constructor que inicializa el skybox con los datos de la malla y el shader.
- [~Skybox](#) ()
Destructor que libera la textura del skybox.
- void [setTexture](#) (GLuint textureID)
Establece la textura del skybox.
- GLuint [getTextureID](#) ()
Obtiene el identificador de la textura del skybox.
- void [render](#) (const glm::mat4 &view, const glm::mat4 &projection)
Renderiza el skybox usando las matrices de vista y proyección proporcionadas.

Atributos privados

- [Mesh](#) mesh
Malla que representa el skybox.
- [ShaderProgram](#) * shader
Shader usado para renderizar el skybox.
- GLuint [textureID](#)
Identificador de la textura del skybox.

6.15.1. Descripción detallada

Clase que representa un skybox para el entorno 3D.

Maneja la carga, asignación y renderizado de la textura del skybox.

6.15.2. Documentación de constructores y destructores

6.15.2.1. `Skybox()`

```
udit::Skybox::Skybox (  
    MeshData mesh,  
    ShaderProgram * shader)
```

Constructor que inicializa el skybox con los datos de la malla y el shader.

Parámetros

mesh	Datos de la malla que representa el skybox.
shader	Puntero al shader que se usará para renderizar el skybox.

6.15.2.2. ~Skybox()

```
udit::Skybox::~~Skybox ()
```

Destructor que libera la textura del skybox.

6.15.3. Documentación de funciones miembro

6.15.3.1. getTextureID()

```
GLuint udit::Skybox::getTextureID ()
```

Obtiene el identificador de la textura del skybox.

Devuelve

El ID de la textura.

6.15.3.2. render()

```
void udit::Skybox::render (
    const glm::mat4 & view,
    const glm::mat4 & projection)
```

Renderiza el skybox usando las matrices de vista y proyección proporcionadas.

Parámetros

view	Matriz de vista.
projection	Matriz de proyección.

6.15.3.3. setTexture()

```
void udit::Skybox::setTexture (
    GLuint textureID)
```

Establece la textura del skybox.

Parámetros

textureID	Identificador de la textura de cubemap.
-----------	---

6.15.4. Documentación de datos miembro

6.15.4.1. mesh

[Mesh](#) udit::Skybox::mesh [private]

Malla que representa el skybox.

6.15.4.2. shader

[ShaderProgram*](#) udit::Skybox::shader [private]

Shader usado para renderizar el skybox.

6.15.4.3. textureID

GLuint udit::Skybox::textureID [private]

Identificador de la textura del skybox.

La documentación de esta clase está generada de los siguientes archivos:

- [Skybox.hpp](#)
- [Skybox.cpp](#)

6.16. Referencia de la clase udit::TextureLoader

Clase para cargar texturas 2D y cubemaps en OpenGL.

```
#include <TextureLoader.hpp>
```

Métodos públicos

- [TextureLoader](#) ()
Constructor por defecto.
- [~TextureLoader](#) ()
Destructor.
- GLuint [loadTexture](#) (const string &filePath)
Carga una textura 2D desde un archivo.
- GLuint [loadCubemap](#) (const vector< string > &faces)
Carga un cubemap desde 6 rutas de archivo (una por cada cara).

Atributos privados

- GLuint [textureID](#)

6.16.1. Descripción detallada

Clase para cargar texturas 2D y cubemaps en OpenGL.

Utiliza stb_image para leer imágenes desde disco y cargarlas como texturas compatibles con OpenGL. Soporta tanto texturas 2D estándar como cubemaps.

6.16.2. Documentación de constructores y destructores

6.16.2.1. TextureLoader()

```
udit::TextureLoader::TextureLoader ()
```

Constructor por defecto.

Inicializa el identificador de textura a 0.

6.16.2.2. ~TextureLoader()

```
udit::TextureLoader::~~TextureLoader ()
```

Destructor.

Libera la textura si ha sido creada.

6.16.3. Documentación de funciones miembro

6.16.3.1. loadCubemap()

```
GLuint udit::TextureLoader::loadCubemap (
    const vector< string > & faces)
```

Carga un cubemap desde 6 rutas de archivo (una por cada cara).

Las caras deben estar ordenadas según GL_TEXTURE_CUBE_MAP_POSITIVE_X + i.

Parámetros

faces	Vector con las rutas a las imágenes de cada cara.
-------	---

Devuelve

GLuint Identificador del cubemap generado en OpenGL. Devuelve 0 si hay error.

6.16.3.2. loadTexture()

```
GLuint udit::TextureLoader::loadTexture (
    const string & filePath)
```

Carga una textura 2D desde un archivo.

Parámetros

filePath	Ruta al archivo de imagen.
----------	----------------------------

Devuelve

GLuint Identificador de la textura generada en OpenGL. Devuelve 0 si hay error.

6.16.4. Documentación de datos miembro

6.16.4.1. textureID

GLuint udit::TextureLoader::textureID [private]

Identificador de la textura generada en OpenGL.

La documentación de esta clase está generada de los siguientes archivos:

- [TextureLoader.hpp](#)
- [TextureLoader.cpp](#)

6.17. Referencia de la clase udit::Window

Clase que encapsula una ventana con soporte para contexto OpenGL y control de cámara.

```
#include <Window.hpp>
```

Clases

- struct [OpenGL_Context_Settings](#)
Estructura para configurar los parámetros del contexto de OpenGL.

Tipos públicos

- enum [Position](#) { [UNDEFINED](#) = SDL_WINDOWPOS_UNDEFINED , [CENTERED](#) = SDL_WINDOWPOS_CENTERED }
- Enumeración para definir la posición inicial de la ventana.

Métodos públicos

- `Window` (`const char *title`, `int left_x`, `int top_y`, `unsigned width`, `unsigned height`, `const OpenGL_Context_Settings &context_details`)
Constructor alternativo usando `const char*` como título.
- `~Window` ()
Destructor de la ventana. Libera el contexto y recursos SDL.
- `Window` (`const Window &`)=`delete`
Constructor de copia eliminado (no se permite copiar ventanas).
- `Window & operator=` (`const Window &`)=`delete`
Operador de copia eliminado.
- `void swap_buffers` ()
Intercambia los buffers del contexto OpenGL.
- `void poll_input_events` (`bool *exit`)
Procesa eventos de entrada SDL como movimiento del ratón y salida del programa.
- `void move_camera` (`bool *exit`)
Procesa eventos de entrada para mover la cámara o salir.
- `Camera get_camera` ()
Obtiene una copia de la cámara actual.

Atributos privados

- `SDL_Window * window_handle`
- `SDL_GLContext opengl_context`
- `Camera camera`

6.17.1. Descripción detallada

Clase que encapsula una ventana con soporte para contexto OpenGL y control de cámara.

Esta clase se encarga de crear y manejar una ventana SDL, configurar el contexto de OpenGL, controlar el ciclo de eventos básicos (ratón y teclado) y actualizar la cámara asociada.

6.17.2. Documentación de las enumeraciones miembro de la clase

6.17.2.1. Position

enum `udit::Window::Position`

Enumeración para definir la posición inicial de la ventana.

Valores de enumeraciones

UNDEFINED	La posición de la ventana no está definida.
CENTERED	La ventana se centrará en la pantalla.

6.17.3. Documentación de constructores y destructores

6.17.3.1. Window() [1/2]

```
udit::Window::Window (  
    const char * title,  
    int left_x,  
    int top_y,  
    unsigned width,  
    unsigned height,  
    const OpenGL\_Context\_Settings & context_details)
```

Constructor alternativo usando const char* como título.

6.17.3.2. ~Window()

```
udit::Window::~~Window ()
```

Destructor de la ventana. Libera el contexto y recursos SDL.

6.17.3.3. Window() [2/2]

```
udit::Window::Window (  
    const Window & ) [delete]
```

Constructor de copia eliminado (no se permite copiar ventanas).

6.17.4. Documentación de funciones miembro

6.17.4.1. get_camera()

```
Camera udit::Window::get_camera ()
```

Obtiene una copia de la cámara actual.

Devuelve

Objeto [Camera](#) con la configuración actual.

6.17.4.2. move_camera()

```
void udit::Window::move_camera (  
    bool * exit)
```

Procesa eventos de entrada para mover la cámara o salir.

Parámetros

exit	Puntero a un booleano que será puesto en true si se solicita salir (evento SDL_QUIT).
------	---

6.17.4.3. operator=()

```
Window & udit::Window::operator= (
    const Window & ) [delete]
```

Operador de copia eliminado.

6.17.4.4. poll_input_events()

```
void udit::Window::poll_input_events (
    bool * exit)
```

Procesa eventos de entrada SDL como movimiento del ratón y salida del programa.

Esta función recorre la cola de eventos SDL y maneja los eventos relevantes:

- Si el ratón se mueve, actualiza la orientación de la cámara.
- Si el usuario solicita cerrar la ventana (evento SDL_QUIT), activa la bandera de salida.

Parámetros

exit	Puntero a una bandera booleana que se establecerá en true si se recibe un evento de salida.
------	---

6.17.4.5. swap_buffers()

```
void udit::Window::swap_buffers ()
```

Intercambia los buffers del contexto OpenGL.

Este método debe llamarse al final de cada frame para mostrar el contenido renderizado.

6.17.5. Documentación de datos miembro

6.17.5.1. camera

```
Camera udit::Window::camera [private]
```

Cámara utilizada para navegar la escena 3D.

6.17.5.2. `opengl_context`

`SDL_GLContext` `udit::Window::opengl_context` [private]

Contexto OpenGL asociado con la ventana.

6.17.5.3. `window_handle`

`SDL_Window*` `udit::Window::window_handle` [private]

Manejador de la ventana SDL.

La documentación de esta clase está generada de los siguientes archivos:

- [Window.hpp](#)
- [Window.cpp](#)

6.18. Referencia de la clase `Window`

Clase que encapsula una ventana con soporte para contexto OpenGL y control de cámara.

`#include <Window.hpp>`

Clases

- struct [OpenGL_Context_Settings](#)
Estructura para configurar los parámetros del contexto de OpenGL.

Tipos públicos

- enum [Position](#) { [UNDEFINED](#) = `SDL_WINDOWPOS_UNDEFINED` , [CENTERED](#) = `SDL_WINDOWPOS_CENTERED` }
- Enumeración para definir la posición inicial de la ventana.

Métodos públicos

- [Window](#) (`const char *title`, `int left_x`, `int top_y`, `unsigned width`, `unsigned height`, `const OpenGL_Context_Settings &context_details`)
Constructor alternativo usando `const char*` como título.
- [Window](#) (`const Window &`)=`delete`
Constructor de copia eliminado (no se permite copiar ventanas).
- [~Window](#) ()
Destructor de la ventana. Libera el contexto y recursos SDL.
- [Window & operator=](#) (`const Window &`)=`delete`
Operador de copia eliminado.
- void [swap_buffers](#) ()
Intercambia los buffers del contexto OpenGL.
- void [poll_input_events](#) (`bool *exit`)
Procesa eventos de entrada SDL como movimiento del ratón y salida del programa.
- void [move_camera](#) (`bool *exit`)
Procesa eventos de entrada para mover la cámara o salir.
- Camera [get_camera](#) ()
Obtiene una copia de la cámara actual.

Atributos privados

- SDL_Window * [window_handle](#)
- SDL_GLContext [opengl_context](#)
- Camera [camera](#)

6.18.1. Descripción detallada

Clase que encapsula una ventana con soporte para contexto OpenGL y control de cámara.

Esta clase se encarga de crear y manejar una ventana SDL, configurar el contexto de OpenGL, controlar el ciclo de eventos básicos (ratón y teclado) y actualizar la cámara asociada.

6.18.2. Documentación de las enumeraciones miembro de la clase

6.18.2.1. Position

enum [udit::Window::Position](#)

Enumeración para definir la posición inicial de la ventana.

Valores de enumeraciones

UNDEFINED	La posición de la ventana no está definida.
CENTERED	La ventana se centrará en la pantalla.

6.18.3. Documentación de constructores y destructores

6.18.3.1. Window() [1/2]

```
udit::Window::Window (
    const char * title,
    int left_x,
    int top_y,
    unsigned width,
    unsigned height,
    const OpenGL\_Context\_Settings & context_details)
```

Constructor alternativo usando const char* como título.

6.18.3.2. Window() [2/2]

```
udit::Window::Window (
    const Window & ) [delete]
```

Constructor de copia eliminado (no se permite copiar ventanas).

6.18.3.3. ~Window()

```
udit::Window::~~Window ()
```

Destructor de la ventana. Libera el contexto y recursos SDL.

6.18.4. Documentación de funciones miembro

6.18.4.1. get_camera()

```
Camera udit::Window::get_camera ()
```

Obtiene una copia de la cámara actual.

Devuelve

Objeto Camera con la configuración actual.

6.18.4.2. move_camera()

```
void udit::Window::move_camera (
    bool * exit)
```

Procesa eventos de entrada para mover la cámara o salir.

Parámetros

exit	Puntero a un booleano que será puesto en true si se solicita salir (evento SDL_QUIT).
------	---

6.18.4.3. operator=()

```
Window & udit::Window::operator= (
    const Window & ) [delete]
```

Operador de copia eliminado.

6.18.4.4. poll_input_events()

```
void udit::Window::poll_input_events (
    bool * exit)
```

Procesa eventos de entrada SDL como movimiento del ratón y salida del programa.

Esta función recorre la cola de eventos SDL y maneja los eventos relevantes:

- Si el ratón se mueve, actualiza la orientación de la cámara.
- Si el usuario solicita cerrar la ventana (evento SDL_QUIT), activa la bandera de salida.

Parámetros

exit	Puntero a una bandera booleana que se establecerá en true si se recibe un evento de salida.
------	---

6.18.4.5. swap_buffers()

```
void udit::Window::swap_buffers ()
```

Intercambia los buffers del contexto OpenGL.

Este método debe llamarse al final de cada frame para mostrar el contenido renderizado.

6.18.5. Documentación de datos miembro

6.18.5.1. camera

```
Camera udit::Window::camera [private]
```

Cámara utilizada para navegar la escena 3D.

6.18.5.2. opengl_context

```
SDL_GLContext udit::Window::opengl_context [private]
```

Contexto OpenGL asociado con la ventana.

6.18.5.3. window_handle

```
SDL_Window* udit::Window::window_handle [private]
```

Manejador de la ventana SDL.

La documentación de esta clase está generada de los siguientes archivos:

- [Window.hpp](#)
- [Window.cpp](#)

Capítulo 7

Documentación de archivos

7.1. Referencia del archivo AssimpMesh.hpp

```
#include "Mesh.hpp"  
#include <assimp/Importer.hpp>  
#include <assimp/scene.h>  
#include <assimp/postprocess.h>
```

Clases

- class [udit::AssimpMesh](#)
Clase que extiende [Mesh](#) para cargar modelos 3D usando Assimp.

Espacios de nombres

- namespace [udit](#)

7.2. AssimpMesh.hpp

[Ir a la documentación de este archivo.](#)

```
00001 #pragma once  
00002 #include "Mesh.hpp"  
00003 #include <assimp/Importer.hpp>  
00004 #include <assimp/scene.h>  
00005 #include <assimp/postprocess.h>  
00006  
00007 namespace udit  
00008 {  
00016     class AssimpMesh : public Mesh  
00017     {  
00018     public:  
00023         AssimpMesh(const std::string& filepath);  
00024  
00025     private:  
00032         void loadModel(const std::string& path);  
00033  
00038         void processMesh(aiMesh* mesh);  
00039  
00044         void processVertices(aiMesh* mesh);  
00045  
00050         void processIndices(aiMesh* mesh);  
00051     };  
00052 }
```

7.3. Referencia del archivo Camera.hpp

```
#include <glm.hpp>
#include <gtc/matrix_transform.hpp>
#include <SDL.h>
```

Clases

- class `udit::Camera`
Clase que representa una cámara en un espacio 3D, controlada mediante teclado y ratón.

Espacios de nombres

- namespace `udit`

7.4. Camera.hpp

[Ir a la documentación de este archivo.](#)

```
00001 #pragma once
00002
00003 #include <glm.hpp>
00004 #include <gtc/matrix_transform.hpp>
00005 #include <SDL.h>
00006
00007 namespace udit
00008 {
00015     class Camera
00016     {
00017     private:
00018         glm::vec3 position;
00019         glm::vec3 front;
00020         glm::vec3 up;
00021         glm::vec3 right;
00022         glm::vec3 world_up;
00023
00024         float yaw;
00025         float pitch;
00026         float movement_speed;
00027         float mouse_sensitivity;
00028
00029     public:
00038         Camera(
00039             glm::vec3 start_position,
00040             glm::vec3 start_up,
00041             float start_yaw,
00042             float start_pitch
00043         );
00044
00048         Camera();
00049
00055         glm::mat4 get_view_matrix() const;
00056
00062         glm::vec3 get_position() const;
00063
00069         void process_keyboard(const Uint8* state);
00070
00076         void start_camera_control();
00077
00084         void process_mouse_motion(float xrel, float yrel);
00085
00086     private:
00092         void update_camera_vectors();
00093     };
00094 }
```


7.5. Referencia del archivo Cone.hpp

```
#include "Mesh.hpp"
```

Clases

- class `udit::Cone`
Representa un cono 3D con un número de divisiones, radio y altura.

Espacios de nombres

- namespace `udit`

7.6. Cone.hpp

[Ir a la documentación de este archivo.](#)

```
00001 /*@file Cone.hpp
00002 * @author andrmatgonros@gmail.com
00003 * @date 2025-01-12
00004 *
00005 * Este código es de dominio público.
00006 *
00007 * @brief Definición de la clase Cone que representa un cono 3D.
00008 *
00009 * La clase Cone se utiliza para generar y renderizar un cono 3D en OpenGL. Permite
00010 * especificar el número de divisiones, el radio de la base y la altura del cono.
00011 * También gestiona los VBOs y VAOs necesarios para la representación del cono.
00012 */
00013
00014 #pragma once
00015 #include "Mesh.hpp"
00016
00017 namespace udit
00018 {
00019     class Cone : public Mesh
00020     {
00021     public:
00022         Cone(int d, GLfloat r, GLfloat h);
00023         ~Cone();
00024
00025     private:
00026         int divisions;
00027         GLfloat radius;
00028         GLfloat height;
00029
00030         void generateGeometry();
00031         void generateApexVertex(int& vertexIndex, int apexIndex);
00032         void generateBaseCenterVertex(int& vertexIndex, int baseCenterIndex);
00033         void generateBaseVertices(int& vertexIndex);
00034     };
00035 }
```

7.7. Referencia del archivo GeometryGenerator.hpp

```
#include <vector>
#include <glm.hpp>
#include <glad/glad.h>
#include <numbers>
```

Clases

- struct [udit::MeshData](#)
Estructura que contiene datos de una malla generada.
- class [udit::GeometryGenerator](#)
Clase para generar primitivas geométricas básicas.

Espacios de nombres

- namespace [udit](#)

7.8. GeometryGenerator.hpp

[Ir a la documentación de este archivo.](#)

```

00001 #pragma once
00002 #include <vector>
00003 #include <glm.hpp>
00004 #include <glad/glad.h>
00005 #include <numbers>
00006
00007 namespace udit
00008 {
00009
00015     struct MeshData
00016     {
00017         std::vector<GLfloat> coordinates;
00018         std::vector<GLfloat> texCoords;
00019         std::vector<GLfloat> normals;
00020         std::vector<GLuint> indices;
00021     };
00022
00028     class GeometryGenerator
00029     {
00030     public:
00039         static MeshData generatePlane(int hDivisions, int vDivisions, float width, float height);
00040
00048         static MeshData generateCone(float radius, float height, int segments);
00049
00055         static MeshData generateCube(float size = 1.0f);
00056     };
00057
00058 }
```

7.9. Referencia del archivo Mesh.hpp

```

#include <glad/glad.h>
#include <vector>
#include <cmath>
#include <numbers>
#include "GeometryGenerator.hpp"
#include "ShaderProgram.hpp"
```

Clases

- class [udit::Mesh](#)
Clase que representa una malla 3D y gestiona sus buffers OpenGL.

Espacios de nombres

- namespace [udit](#)

7.10. Mesh.hpp

[Ir a la documentación de este archivo.](#)

```

00001 #pragma once
00002 #include <glad/glad.h>
00003 #include <vector>
00004 #include <cmath>
00005 #include <numbers>
00006 #include "GeometryGenerator.hpp"
00007 #include "ShaderProgram.hpp"
00008 using namespace std;
00009
00010 namespace udit
00011 {
00012     class Mesh
00013     {
00014     public:
00025         Mesh();
00026
00033         Mesh(MeshData receivedData);
00034
00041         void render();
00042
00049         void generateBuffers();
00050
00059         void uploadBuffer(GLuint bufferId, GLenum target, const void* dataPtr, size_t dataSize);
00060
00066         void deleteBuffers();
00067
00068     protected:
00069
00071         enum { COORDINATES_VBO, TEXCOORDS_VBO, NORMALS_VBO, INDICES_EBO, VBO_COUNT };
00072
00074         bool isInitialized;
00075
00077         GLuint vao_id;
00078
00080         GLuint vbo_ids[VBO_COUNT];
00081
00083         MeshData data;
00084     };
00085 }

```

7.11. Referencia del archivo Object.hpp

```
#include "Mesh.hpp"
```

Clases

- class [udit::Object](#)
Representa un objeto 3D con una malla, shader y transformaciones.

Espacios de nombres

- namespace [udit](#)

7.12. Object.hpp

[Ir a la documentación de este archivo.](#)

```
00001 #pragma once
00002 #include "Mesh.hpp"
00003
00004 namespace udit
00005 {
00011     class Object {
00012     public:
00018         Object(Mesh* mesh, ShaderProgram* shader);
00019
00027         void render(const glm::mat4& view, const glm::mat4& projection);
00028
00034         void setupTexturesAndBlending();
00035
00040         void bindHeightmapAndTexture();
00041
00046         void bindTextureWithTransparency();
00047
00051         void bindTextureOnly();
00052
00057         void cleanupBlending();
00058
00059
00064         void setPosition(const glm::vec3& pos);
00065
00070         void setRotation(const glm::vec3& rot);
00071
00076         void setScale(const glm::vec3& scl);
00077
00082         void setTextureID(GLuint texture);
00083
00088         void setHeightmapTextureID(GLuint texture);
00089
00094         ShaderProgram* getShader() const;
00095
00100         Mesh* getMesh() const;
00101
00102     private:
00103         Mesh* mesh;
00104         ShaderProgram* shader;
00105
00106         GLuint textureID;
00107         GLuint heightmapID;
00108
00109         GLint heightmapLoc;
00110         GLint transparencyLoc;
00111
00112         glm::vec3 position;
00113         glm::vec3 rotation;
00114         glm::vec3 scale;
00115
00120         glm::mat4 computeModelMatrix() const;
00121     };
00122 }
```

7.13. Referencia del archivo Plane.hpp

```
#include "Mesh.hpp"
```

Clases

- class [udit::Plane](#)
Clase para representar un plano 3D.

Espacios de nombres

- namespace [udit](#)

7.14. Plane.hpp

[Ir a la documentación de este archivo.](#)

```
00001 /*@file Plane.hpp
00002 * @author andrmatgonros@gmail.com
00003 * @date 2025-01-12
00004 *
00005 * Este código es de dominio público.
00006 *
00007 * @brief Declaración de la clase Plane para representar un plano 3D.
00008 *
00009 * Esta clase genera un plano de malla utilizando OpenGL, donde el plano tiene una
00010 * cuadrícula definida por su ancho y alto. Se utiliza para representar superficies planas.
00011 */
00012
00013 #pragma once
00014 #include "Mesh.hpp"
00015 using namespace std;
00016
00017 namespace udit
00018 {
00026     class Plane : public Mesh
00027     {
00028     public:
00038         Plane(int subdivisionsX, int subdivisionsY, float width, float height);
00039
00045         ~Plane();
00046
00047     private:
00048
00049         float grid_width;
00050         float grid_height;
00051         int subdivX, subdivY;
00058         void generateGeometry();
00059     };
00060 }
```

7.15. Referencia del archivo Scene.hpp

Declaración de la clase [Scene](#) que representa una escena 3D completa.

```
#include "AssimpMesh.hpp"
#include "Camera.hpp"
#include "TextureLoader.hpp"
#include "Skybox.hpp"
#include "SceneNode.hpp"
#include <cassert>
#include <SDL.h>
```

Clases

- class [udit::Scene](#)
Representa una escena 3D con cámara, objetos, skybox, texturas, iluminación y shaders.

Espacios de nombres

- namespace [udit](#)

7.15.1. Descripción detallada

Declaración de la clase [Scene](#) que representa una escena 3D completa.

7.16. Scene.hpp

[Ir a la documentación de este archivo.](#)

```

00001
00005
00006 #pragma once
00007
00008 #include "AssimpMesh.hpp"
00009 #include "Camera.hpp"
00010 #include "TextureLoader.hpp"
00011 #include "Skybox.hpp"
00012 #include "SceneNode.hpp"
00013 #include <cassert>
00014 #include <SDL.h>
00015
00016 namespace udit
00017 {
00026     class Scene
00027     {
00028     private:
00029
00030         GeometryGenerator generator;
00031
00032         unique_ptr<ShaderProgram> defaultProgram;
00033         unique_ptr<ShaderProgram> unlitProgram;
00034         unique_ptr<ShaderProgram> skyboxProgram;
00035         unique_ptr<ShaderProgram> heightmapProgram;
00036
00037         static const float UFO_HEIGHT;
00038         static const float COW_HEIGHT;
00039         static const float CONE_HEIGHT;
00040         static const float SCALE_SMALL;
00041         static const float SCALE_BIG;
00042
00043         glm::vec3 lightPos;
00044         glm::vec3 lightColor;
00045         glm::vec3 viewPos;
00046
00047         glm::mat4 projection_matrix;
00048
00049
00050         unique_ptr<AssimpMesh> ufo;
00051         unique_ptr<AssimpMesh> cow;
00052
00053         unique_ptr<Mesh> plane;
00054         unique_ptr<Mesh> cone;
00055
00056         unique_ptr<Object> heightmapObj;
00057         unique_ptr<Object> ufoObj;
00058         unique_ptr<Object> cowObj;
00059         unique_ptr<Object> coneObj;
00060         unique_ptr<Skybox> skybox;
00061
00062         float angle;
00063
00064         Camera camera;
00065
00066         TextureLoader textureLoader;
00067
00068         GLuint coneTextureID;
00069         GLuint skyboxTextureID;
00070         GLuint heightmapID;
00071         GLuint heightmapTextureID;
00072         GLuint cowTextureID;
00073         GLuint ufoTextureID;
00074
00075         std::shared_ptr<SceneNode> rootNode;
00076         std::shared_ptr<SceneNode> ufoCowConeNode;
00077         std::shared_ptr<SceneNode> ufoNode;
00078         std::shared_ptr<SceneNode> cowNode;
00079         std::shared_ptr<SceneNode> coneNode;
00080         std::shared_ptr<SceneNode> heightmapNode;
00081
00082
00083         void initResources();
00084         void initObjects();
00085         void initSceneGraph();
00086     public:
00094         Scene(unsigned width, unsigned height);
00095
00096         ~Scene();
00097
00101         void loadTextures();
00102
00106         void setTextures();

```

```

00107
00111     void update();
00112
00116     void render();
00117
00118     void applyLightSettings(ShaderProgram& shader, const vector<glm::vec3>& dirsView, const vector<glm::vec3>&
colors, const vector<float>& intensities);
00119
00124     void lightSetup(glm::mat4& view_matrix);
00125
00131     void resize(unsigned width, unsigned height);
00132
00137     void set_camera(Camera new_camera);
00138 };
00139 }

```

7.17. Referencia del archivo SceneNode.hpp

```

#include "Object.hpp"
#include <vector>
#include <memory>
#include <gtc/matrix_transform.hpp>

```

Clases

- class `udit::SceneNode`
Nodo de escena para jerarquía de objetos 3D.

Espacios de nombres

- namespace `udit`

7.18. SceneNode.hpp

[Ir a la documentación de este archivo.](#)

```

00001 #pragma once
00002 #include "Object.hpp"
00003 #include <vector>
00004 #include <memory>
00005 #include <gtc/matrix_transform.hpp>
00006
00007 namespace udit
00008 {
00009
00017     class SceneNode
00018     {
00019     public:
00024         SceneNode(Object* object = nullptr);
00025
00030         void addChild(std::shared_ptr<SceneNode> child);
00031
00038         void setTransform(const glm::vec3& position, const glm::vec3& rotation, const glm::vec3& scale);
00039
00048         void render(const glm::mat4& parentTransform, const glm::mat4& view, const glm::mat4& projection);
00049
00050     private:
00051         Object* object;
00052         glm::vec3 position;
00053         glm::vec3 rotation;
00054         glm::vec3 scale;
00055
00056         std::vector<std::shared_ptr<SceneNode>> children;
00057
00062         glm::mat4 computeLocalTransform() const;
00063     };
00064
00065 }

```

7.19. Referencia del archivo ShaderProgram.hpp

```
#include <glad/glad.h>
#include <glm.hpp>
#include <gtc/matrix_transform.hpp>
#include <gtc/type_ptr.hpp>
#include <iostream>
#include <string>
#include <unordered_map>
```

Clases

- class [udit::ShaderProgram](#)
Clase para manejar programas de sombreado (shaders) en OpenGL.

Espacios de nombres

- namespace [udit](#)

7.20. ShaderProgram.hpp

[Ir a la documentación de este archivo.](#)

```
00001 #pragma once
00002
00003 #include <glad/glad.h>
00004 #include <glm.hpp>
00005 #include <gtc/matrix_transform.hpp>
00006 #include <gtc/type_ptr.hpp>
00007 #include <iostream>
00008 #include <string>
00009 #include <unordered_map>
00010
00011 using namespace std;
00012
00013 namespace udit
00014 {
00020     class ShaderProgram
00021     {
00022     private:
00023         GLuint program_id;
00024
00025         mutable std::unordered_map<std::string, GLint> uniform_locations;
00030         void show_compilation_error(GLuint shader_id) const;
00031
00036         void show_linkage_error(GLuint program_id) const;
00037
00043         GLint getUniformLocation(const std::string& name) const;
00044
00045     public:
00051         ShaderProgram(const std::string& vertex_path, const std::string& fragment_path);
00052
00056         ~ShaderProgram();
00057
00058         ShaderProgram(const ShaderProgram&) = delete;
00059         ShaderProgram& operator=(const ShaderProgram&) = delete;
00060
00065         ShaderProgram(ShaderProgram&& other) noexcept;
00066
00072         ShaderProgram& operator=(ShaderProgram&& other) noexcept;
00076         void use() const;
00077
00082         GLuint id() const { return program_id; }
00083
00089         void setMat4(const std::string& name, const glm::mat4& mat) const;
00090
00096         void setVec3(const std::string& name, const glm::vec3& vec) const;
00097
00103         void setFloat(const std::string& name, float value) const;
00104
00110         void setInt(const std::string& name, int value) const;
00111     };
00112 }
```


7.21. Referencia del archivo Skybox.hpp

```
#include <string>
#include <vector>
#include <glad/glad.h>
#include <gtc/type_ptr.hpp>
#include <glm.hpp>
#include <iostream>
#include <stb_image.h>
#include "Mesh.hpp"
#include "TextureLoader.hpp"
```

Clases

- class [udit::Skybox](#)
Clase que representa un skybox para el entorno 3D.

Espacios de nombres

- namespace [udit](#)

7.22. Skybox.hpp

[Ir a la documentación de este archivo.](#)

```
00001 #pragma once
00002
00003 #include <string>
00004 #include <vector>
00005 #include <glad/glad.h>
00006 #include <gtc/type_ptr.hpp>
00007 #include <glm.hpp>
00008 #include <iostream>
00009 #include <stb_image.h>
00010 #include "Mesh.hpp"
00011 #include "TextureLoader.hpp"
00012
00013 namespace udit
00014 {
00021     class Skybox
00022     {
00023     public:
00024
00030         Skybox(MeshData mesh, ShaderProgram* shader);
00031
00035         ~Skybox();
00036
00041         void setTexture(GLuint textureID);
00042
00047         GLuint getTextureID();
00048
00054         void render(const glm::mat4& view, const glm::mat4& projection);
00055
00056     private:
00057         Mesh mesh;
00058         ShaderProgram* shader;
00059
00060         GLuint textureID;
00061     };
00062 }
```

7.23. Referencia del archivo TextureLoader.hpp

```
#include <string>
#include <vector>
#include <glad/glad.h>
#include <iostream>
#include <stb_image.h>
```

Clases

- class [udit::TextureLoader](#)
Clase para cargar texturas 2D y cubemaps en OpenGL.

Espacios de nombres

- namespace [udit](#)

7.24. TextureLoader.hpp

[Ir a la documentación de este archivo.](#)

```
00001 #pragma once
00002
00003 #include <string>
00004 #include <vector>
00005 #include <glad/glad.h>
00006 #include <iostream>
00007 #include <stb_image.h>
00008
00009 using namespace std;
00010
00011 namespace udit
00012 {
00013     class TextureLoader
00014     {
00015     public:
00016         TextureLoader();
00017         ~TextureLoader();
00018
00019         GLuint loadTexture(const string& filePath);
00020         GLuint loadCubemap(const vector<string>& faces);
00021     private:
00022         GLuint textureID;
00023     };
00024 }
```

7.25. Referencia del archivo Window.hpp

```
#include <SDL.h>
#include <string>
#include <utility>
#include "Camera.hpp"
```

Clases

- class `udit::Window`
Clase que encapsula una ventana con soporte para contexto OpenGL y control de cámara.
- struct `udit::Window::OpenGL_Context_Settings`
Estructura para configurar los parámetros del contexto de OpenGL.

Espacios de nombres

- namespace `udit`

7.26. Window.hpp

[Ir a la documentación de este archivo.](#)

```

00001 #pragma once
00002
00003 #include <SDL.h>
00004 #include <string>
00005 #include <utility>
00006 #include "Camera.hpp"
00007
00008 namespace udit
00009 {
00010     class Window
00011     {
00012     public:
00013         enum Position
00014         {
00015             UNDEFINED = SDL_WINDOWPOS_UNDEFINED,
00016             CENTERED = SDL_WINDOWPOS_CENTERED,
00017         };
00018
00019         struct OpenGL_Context_Settings
00020         {
00021             unsigned version_major = 3;
00022             unsigned version_minor = 3;
00023             bool core_profile = true;
00024             unsigned depth_buffer_size = 24;
00025             unsigned stencil_buffer_size = 0;
00026             bool enable_vsync = true;
00027         };
00028
00029     private:
00030         SDL_Window* window_handle;
00031         SDL_GLContext opengl_context;
00032         Camera camera;
00033
00034     public:
00035         Window(
00036             const char* title,
00037             int left_x,
00038             int top_y,
00039             unsigned width,
00040             unsigned height,
00041             const OpenGL_Context_Settings& context_details
00042         );
00043
00044         ~Window();
00045
00046         Window(const Window&) = delete;
00047
00048         Window& operator = (const Window&) = delete;
00049
00050         void swap_buffers();
00051
00052         void poll_input_events(bool* exit);
00053
00054         void move_camera(bool* exit);
00055
00056         Camera get_camera();
00057     };
00058 }
00059
00060 }
```

7.27. Referencia del archivo AssimpMesh.cpp

```
#include "../Headers/AssimpMesh.hpp"  
#include <iostream>
```

Espacios de nombres

- namespace [udit](#)

7.28. Referencia del archivo Camera.cpp

```
#include "../Headers/Camera.hpp"
```

Espacios de nombres

- namespace [udit](#)

7.29. Referencia del archivo Cone.cpp

```
#include "../Headers/Cone.hpp"
```

Espacios de nombres

- namespace [udit](#)

7.30. Referencia del archivo GeometryGenerator.cpp

```
#include "../Headers/GeometryGenerator.hpp"
```

Espacios de nombres

- namespace [udit](#)

Funciones

- static void [udit::addVertex](#) ([MeshData](#) &data, const glm::vec3 &pos, const glm::vec3 &normal, const glm::vec2 &uv)

7.31. Referencia del archivo main.cpp

```
#include "../Headers/Scene.hpp"  
#include "../Headers/Window.hpp"
```

Clases

- class [Scene](#)
Representa una escena 3D con cámara, objetos, skybox, texturas, iluminación y shaders.
- class [Window](#)
Clase que encapsula una ventana con soporte para contexto OpenGL y control de cámara.

Funciones

- int [main](#) (int argc, char *argv[])

7.31.1. Documentación de funciones

7.31.1.1. main()

```
int main (  
    int argc,  
    char * argv[])
```

< Nombre de la ventana.

< Posición centrada de la ventana en la pantalla.

< Posición centrada de la ventana en la pantalla.

< Ancho de la ventana.

< Alto de la ventana.

< Crea la escena 3D con el tamaño de la ventana.

< Flag para determinar si el programa debe salir.

< Actualiza la escena.

< Renderiza la escena.

< Intercambia los buffers para mostrar la imagen renderizada.

< Mueve la cámara

< La cámara de la escena recibe los valores de la cámara de la ventana

< Finaliza SDL al salir del bucle.

7.32. Referencia del archivo Mesh.cpp

```
#include "../Headers/Mesh.hpp"
```

Espacios de nombres

- namespace [udit](#)

7.33. Referencia del archivo Object.cpp

```
#include "../Headers/Object.hpp"
```

Espacios de nombres

- namespace [udit](#)

7.34. Referencia del archivo Plane.cpp

```
#include "../Headers/Plane.hpp"  
#include <vector>
```

Espacios de nombres

- namespace [udit](#)

7.35. Referencia del archivo Scene.cpp

```
#include "../Headers/Scene.hpp"
```

Espacios de nombres

- namespace [udit](#)

7.36. Referencia del archivo SceneNode.cpp

```
#include "../Headers/SceneNode.hpp"
```

Espacios de nombres

- namespace [udit](#)

7.37. Referencia del archivo ShaderProgram.cpp

```
#include "../Headers/ShaderProgram.hpp"  
#include <fstream>  
#include <sstream>  
#include <stdexcept>
```

Espacios de nombres

- namespace [udit](#)

Funciones

- static std::string [readFile](#) (const std::string &filepath)

7.37.1. Documentación de funciones

7.37.1.1. readFile()

```
static std::string readFile (  
    const std::string & filepath) [static]
```

7.38. Referencia del archivo Skybox.cpp

```
#include "../Headers/Skybox.hpp"
```

Espacios de nombres

- namespace [udit](#)

7.39. Referencia del archivo stb_image.cpp

```
#include <stb_image.h>
```

defines

- #define [STB_IMAGE_IMPLEMENTATION](#)

7.39.1. Documentación de «define»

7.39.1.1. STB_IMAGE_IMPLEMENTATION

```
#define STB_IMAGE_IMPLEMENTATION
```

7.40. Referencia del archivo TextureLoader.cpp

```
#include "../Headers/TextureLoader.hpp"
```

Espacios de nombres

- namespace [udit](#)

7.41. Referencia del archivo Window.cpp

```
#include <cassert>
#include <glad/glad.h>
#include <SDL_opengl.h>
#include "../Headers/Window.hpp"
#include <stdexcept>
```

Espacios de nombres

- namespace [udit](#)